International Journal of Production Research

ISSN: 0020-7543 (Print) 1366-588X (Online) Journal homepage: https://www.tandfonline.com/loi/tprs20

An energy-efficient two-stage hybrid flow shop scheduling problem in a glass production

Shijin Wang, Xiaodong Wang, Feng Chu & Jianbo Yu

To cite this article: Shijin Wang, Xiaodong Wang, Feng Chu & Jianbo Yu (2019): An energy-efficient two-stage hybrid flow shop scheduling problem in a glass production, International Journal of Production Research, DOI: 10.1080/00207543.2019.1624857

To link to this article: https://doi.org/10.1080/00207543.2019.1624857

Published online: 07 Jun 2019.

International Journal of Production Research, 2019
https://doi.org/10.1080/00207543.2019.1624857

An energy-efficient two-stage hybrid flow shop scheduling problem in a glass production

Shijin Wang[a*], Xiaodong Wang[a], Feng Chu[b,c] and Jianbo Yu[d]

[a]*School of Economics and Management, Tongji University, Shanghai, People's Republic of China;* [b]*IBISC, Univ Évry, University of Paris-Saclay, Paris, France;* [c]*School of Economics and Management, Fuzhou University, Fuzhou, People's Republic of China;* [d]*School of Mechanical Engineering, Tongji University, Shanghai, People's Republic of China*

(*Received 14 November 2018; accepted 13 May 2019*)

Energy-efficient scheduling is highly necessary for energy-intensive industries, such as glass, mould or chemical production. Inspired by a real-world glass-ceramics production process, this paper investigates a bi-criteria energy-efficient two-stage hybrid flow shop scheduling problem, in which parallel machines with eligibility are at stage 1 and a batch machine is at stage 2. The performance measures considered are makespan and total energy consumption. Time-of-use (TOU) electricity prices and different states of machines (working, idle and turnoff) are integrated. To tackle this problem, a mixed integer programming (MIP) is formulated, based on which an augmented ϵ-constraint (AUGMECON) method is adopted to obtain the exact Pareto front. A problem-tailored constructive heuristic method with local search strategy, a bi-objective tabu search algorithm and a bi-objective ant colony optimisation algorithm are developed to deal with medium- and large-scale problems. Extensive computational experiments are conducted, and a real-world case is solved. The results show effectiveness of the proposed methods, in particular the bi-objective tabu search.

Keywords: two-stage hybrid flow shop; total energy consumption; ϵ-constraint method; constructive heuristic; bi-objective tabu search; bi-objective ant colony optimisation

1. Introduction

Due to the development of global economy and population expansion, energy consumption worldwide has been increasing rapidly. More specifically, the total energy consumption in industrial sector in the world will grow from 58.9 Peta Watt hour (PWh) in 2010 to 90.4 PWh in 2040 (Wang, Wang, et al. 2018). Among all sectors, the industrial sector continues to contribute nearly 50% of the total energy consumption (Wang, Liu, et al. 2016). Moreover, most of resources for energy generation are nonrenewable (Tacconi 2000). In this context, improving energy usage efficiency in manufacturing and other industrial activities, especially for energy-intensive companies, is of urgent importance for both energy saving and environment protection (Giret, Trentesaux, and Prabhu 2015; Merkert et al. 2015; Rager, Gahm, and Denz 2015; Gahm et al. 2016).

As a typical manufacturing environment, hybrid flow shop (HFS) is applied in various production scenarios, including chemicals, electric appliances and other small-batch customised industrial products (Ribas, Leisten, and Framiñan 2010; Ruiz and Vázquez-Rodriguez 2010). Among various scheduling problems in HFS, the two-stage HFS scheduling problem has been proved to be an NP-hard problem (Gupta 1988). Due to its practical and theoretical significance, there are continuous research efforts on two-stage HFS scheduling problems in both academia and industry. However, the problem with energy consumption has not yet been sufficiently explored (Ruiz and Vázquez-Rodriguez 2010).

In this paper, motivated by scheduling challenges of real-world glass decoration and ceramisation, we study a two-stage hybrid flow shop scheduling problem with the consideration of both machine utilisation and energy consumption.

The investigated problem is originated from a glass manufacturer who supplies glass-ceramics for a variety of industries including construction, decorating, automobile and furniture. More specifically, the problem is concentrated on a HFS scheduling problem with multiple parallel machines which have different energy consumption rates at stage 1, and a single batch machine at stage 2. The machine eligibility and the states of machines (working, idle and turnoff) are integrated. The time-of-use (TOU) electricity price, a very common demand-side management strategy, is considered, which means that electricity prices are changed for the time periods of peak, valley and normal. Two objectives, the makespan and the total energy consumption cost, are considered simultaneously for the reasons of: (i) a minimum makespan usually implies a high

*Corresponding author. Email: shijinwang@tongji.edu.cn

utilisation of the equipments, and (ii) energy takes a large portion of the total costs in the company and most electricity consumption cost comes from the machine processing procedure including the parallel machines and the batch machine.

The contributions of this paper include:

(i) A bi-objective two-stage HFS scheduling problem is derived from a real-world glass manufacturing environment. It is formulated as a comprehensive mixed integer programming (MIP) by considering machine eligibility, the states of machines, different energy consumption rates of machines, batch forming and the TOU electricity prices.
(ii) The augmented ϵ-constraint method is adopted to obtain the exact Pareto front. Two other methods: a bi-objective tabu search algorithm and a bi-objective ant colony optimisation algorithm, both integrated with a constructive heuristic method, are proposed to obtain good approximate Pareto fronts in a reasonable computation time.

The rest of the paper is organised as follows. Relevant literatures are reviewed in Section 2. In Section 3, the problem is described in detail and formulated as a MIP model. Section 4 introduces the detailed application of exact augmented ϵ-constraint method for the problem and the development of two meta-heuristic methods for approximate Pareto fronts. In Section 5, computational experiments are conducted and a case study is given, and the computational results are reported. Conclusions and future research directions are given in Section 6.

2. Literature review

Energy-efficient scheduling problems have been studied extensively in recent years. Various manufacturing environments are considered, including single machine (Mouzon and Yildirim 2008; Yildirim and Mouzon 2012; Liu, Yang, et al. 2014; Shrouf et al. 2014; Che, Zeng, and Lyu 2016; Wang, Qiao, et al. 2016; Che et al. 2017; Cheng et al. 2017; Karhi and Shabtay 2017; Aghelinejad, Ouazene, and Yalaoui 2018a, 2018b; Zhang et al. 2018; Rubaiee and Yildirim 2019), parallel machine (Fang and Lin 2013; Ji, Wang, and Lee 2013; Ding, Song, and Wu 2016; Li et al. 2016; Mansouri, Aktas, and Besikci 2016; Che, Zhang, and Wu 2017; Wang, Wang, and Lin 2017; Wang, Wang, et al. 2018; Abikarram, McConky, and Proano 2019; Safarzadeh and Niaki 2019; Wu and Che 2019), flow shop (Jiang and Wang 2019), job shop (Liu, Dong, et al. 2014; May et al. 2015; Zhang and Chiong 2016; Meng et al., 'MILP Models for Energy-aware' 2019b) and flexible job shop (He et al. 2015; Lei, Zheng, and Guo 2017; Mokhtari and Hasani 2017; Nouiri, Bekrar, and Trentesaux 2018; Wang, Jiang, et al. 2018; Wu and Sun 2018; Gong et al. 2019; Liu, Guo, and Wang 2019). The reader is referred to excellent survey papers by Gahm et al. (2016), Rager, Gahm, and Denz (2015), Merkert et al. (2015) and Giret, Trentesaux, and Prabhu (2015) for review of researches on the energy-efficient scheduling problems.

In what follows, we mainly review three streams of researches that are highly related to this paper: (i) scheduling problems under explicit TOU prices, (ii) HFS or flexible flow shop (FFS) scheduling problems with energy considerations, and (iii) batch scheduling with consideration of energy consumption.

2.1. Related works concerning scheduling problem under explicit TOU prices

There are various energy charging policies which are widely used to compute energy costs, like TOU pricing, critical peak pricing, or other time-dependent energy cost (Aghelinejad, Ouazene, and Yalaoui 2018a). Under the TOU tariffs, the electricity price is higher during the peak period, while lower in the off-peak period (Cui and Li 2018). By far, there are many researches on scheduling problems under explicit TOU tariffs.

In the context of single machine, Che, Zeng, and Lyu (2016) developed a new continuous-time MIP model and a greedy insertion heuristic for the scheduling problem under TOU tariffs to minimise the total electricity cost. Fang et al. (2016) investigated a scheduling problem under TOU tariffs to minimise the total electricity cost. Two types of job processing characteristics, uniform-speed case and speed-scalable case, were considered separately. The complexity of non-preemptive versions of these two cases was explored. They also proved that the preemptive versions of these two cases are polynomial-time solvable. Lee et al. (2017) proposed a dynamic control algorithm for a scheduling problem with due dates and under TOU prices. Two objectives, the total penalty costs for earliness and tardiness, and total energy consumption cost were minimised simultaneously. Chen and Zhang (2019) addressed a class of problems that scheduling jobs on a single machine under TOU tariffs. The objective was to minimise the total cost of processing all these jobs, subjecting to a minimum level of performance in one of these regular criteria: bounded lateness, tardiness, delivery- or flow-time, and total completion times. The computational tractability of each problem was established. Rubaiee and Yildirim (2019) studied a multi-objective preemptive single machine scheduling problem to minimise the total completion time and total energy cost under TOU prices. Three variants of bi-objective ant colony optimisation (ACO) algorithms were developed for the problem.

In the context of parallel machine, Tan, Duan, and Su (2018) investigated an integrated economic batch sizing and scheduling problem under TOU tariffs to minimise electricity cost. Che, Zhang, and Wu (2017) developed a two-stage heuristic for an unrelated parallel machine scheduling problem under TOU prices to minimise the total electricity cost. Under TOU tariffs, Zeng, Che, and Wu (2018) investigated a bi-objective scheduling problem on uniform parallel machines to minimise the total electricity cost and the number of machines actually used simultaneously.

Moon and Park (2014) studied a flexible job shop scheduling problem to minimise the total production cost under TOU and critical peak pricing policy. Constraint programming and MIP were applied to solve the problem.

As for flow shop environments, Luo et al. (2013) studied a bi-objective HFS scheduling problem to minimise the electricity consumption cost and the makespan under TOU pricing policy. They proposed a multi-objective ACO algorithm to obtain approximate Pareto fronts. Zhang et al. (2014) investigated a flow shop scheduling problem to minimise electricity cost and the carbon footprint under TOU tariffs without compromising production throughput. A time-indexed MIP was proposed and a case study was conducted. Zhao, Grossmann, and Tang (2018) considered an integrated scheduling problem of rolling sector in steel production with the consideration of electricity consumption under TOU tariffs. The problem was firstly formulated as a continuous time mixed-integer nonlinear programming, which was then linearised into a MIP formulation.

Besides the time-dependent energy cost like TOU tariffs, more and more researches on energy-efficient scheduling have been integrated with machine-dependent energy charges by considering different energy consumption rates of machine states, including working, idle, turnoff and turn-on (Moon and Park 2014; Che, Zhang, and Wu 2017; Cheng et al. 2017; Aghelinejad, Ouazene, and Yalaoui 2018a).

2.2. Related works concerning HFS or FFS scheduling with energy considerations

Bruzzone et al. (2012) addressed an FFS scheduling problem to minimise the weighted sum of the total tardiness and the makespan with the constraint of the shop floor power's peak. A MIP model was formulated and a RAndomised Neighbourhood Search (RANS) method was proposed for the problem. Inspired from a scheduling problem in a metal working workshop, Dai et al. (2013) studied a bi-objective FFS scheduling problem to minimise the makespan and the total energy consumption. They proposed an improved simulated annealing (SA) algorithm for the problem.

Yan et al. (2016) proposed a multi-level optimisation method for FFS to reduce the total energy consumption and makespan. A multi-objective genetic algorithm (GA) was developed. Tang et al. (2016) studied a dynamic FFS scheduling problem to reduce energy consumption and makespan. Four different particle swarm optimisation (PSO) algorithms were developed for the problem. Li et al. (2018) developed a multi-objective optimisation algorithm for the HFS scheduling problem with setup energy consumption to minimise the makespan and the energy consumption simultaneously.

Liu et al. (2018) studied an energy-oriented bi-objective HFS scheduling problem with machine eligibility and setup times to minimise the makespan and the total electricity cost under TOU pricing policy. A model-based heuristic and a bi-objective differential evolution algorithm were devised to tackle this problem. Lu et al. (2018) investigated a multi-stage HFS problem to minimise three objectives: makespan, noise pollution and total energy consumption. A mathematical formulation was proposed, and a new energy saving strategy was presented. To deal with large-scale problems, they designed a hybrid multi-objective grey wolf algorithm with local search strategy.

Nasiri et al. (2018) developed a bi-objective MIP formulation to minimise the total weighted tardiness and the energy consumption in the HFS environment. Two meta-heuristic algorithms were adopted to obtain the approximate Pareto fronts. Meng et al., 'Mathematical Modelling and Optimisation' (2019) investigated a HFS scheduling problem to minimise the total energy consumption with the constraint of makespan and with the different energy consumption of turning on/off states of machines. Several MIP models were formulated and an improved GA was developed.

The literature review above shows that for HFS or FFS scheduling with energy considerations, the efforts so far have been focused more on the developments or applications of multi-objective meta-heuristic methods including multi-objective PSO, multi-objective GA and multi-objective ACO. However, exact methods are not much explored. On the other hand, batch is not considered in the literature mentioned above. Batch incurs more decisions to be made including how to form a batch and when to start processing a batch, which makes the problem more complicated but more practical.

2.3. Related works concerning batch scheduling with the consideration of energy consumption

Recently, there are some researches in batch scheduling with the consideration of energy consumption. Liu (2014) developed an ϵ-archived GA to examine single-machine and parallel-machine batch scheduling problems to minimise the CO_2 emissions and the total weighted tardiness. Wang, Qiao, et al. (2016) investigated a single machine batch scheduling problem in industrial heating treatment with the objective of minimising gas energy consumption and total weighted tardiness.

Fuzzy logic was adopted to characterise the flexible due dates of workpieces, and a non-dominated sorting genetic algorithm (NSGA) was employed. Wang, Liu, et al. (2016) addressed a bi-objective single machine batch scheduling problem to minimise the total energy consumption and the makespan simultaneously. An exact ϵ-constraint method was proposed to obtain the exact Pareto front. Two heuristic methods based on decomposition ideas were designed to obtain approximate Pareto fronts. Cheng et al. (2017) studied a bi-objective single batch machine scheduling problem to minimise makespan and total electricity costs. A bi-objective MIP model was formulated and an ϵ-constraint method was developed to find the Pareto front. Jia et al. (2017) investigated a parallel identical batch-processing machine scheduling problem to minimise the makespan and the total electric power cost simultaneously. A multi-objective ACO algorithm was developed for the problem. Xu, Tang, and Pistikopoulos (2018) studied a steelmaking scheduling problem with batching decisions and energy constraints. A mixed integer non-linear programming was formulated and a spatial branch and bound algorithm was proposed for the problem. Zeng et al. (2018) considered a multi-objective FFS scheduling with batch process to minimise makespan, total electricity consumption and material wastage. A hybrid NSGA-II method was applied to tackle real-world cases. Zhou et al. (2018) investigated a parallel batch processing machine scheduling problem with dynamic job arrivals under TOU pricing, to minimise makespan and total electricity cost simultaneously. A MIP model was formulated, and a multi-objective differential evolution algorithm was developed for the problem.

In this paper, inspired by a real-world scheduling problem in glass production, we study a deterministic bi-objective energy-efficient scheduling problem in a two-stage HFS environment with parallel machines at stage 1 and a single batch machine at stage 2. Both exact and meta-heuristic methods are developed for the problem. To the best of our knowledge, there is no report on this problem yet in the literature.

3. Problem statement and formulation

3.1. Problem description and assumptions

Glass-ceramics is a kind of composite materials combined by crystal phase and glass, they are made through the process of high temperature melting, forming and heating. The schematic manufacturing process of this kind of glass is shown in Figure 1. Once production orders are placed by customers, raw glass already fabricated are cut into the customers' desired sizes. Then, the edges of each glass are smoothed and bevelled. Next, small and large holes are drilled for future controlling knobs installation. After cleaning, customised logos and graphs are printed on the glass immediately followed by drying process. In the last step, each piece of glass is heated to a certain high degree, which makes the glass change at the molecular level and the glass is partially crystallising and the glass-ceramic is formed finally (Wang, Liu, et al. 2016).

The scheduling problem can be described as follows. Two stages are derived and considered. At stage 1, several parallel machines are used to process (e.g. grind, drill or print, etc.) a set of jobs with nonidentical sizes (the sizes of glasses that customer required are not necessary the same), followed by a single batch processing machine (i.e. furnace) for ceramisation at stage 2 in which several jobs can be processed simultaneously in a batch if the total sizes of the jobs do not exceed the size of the single batch processing machine. We have to make sure that every job is finished at stage 1 before it can be delivered to stage 2. There is en eligibility constraint of parallel machines at stage 1, i.e. not all machines are eligible for processing each job, since not all machines can deal with all customised logos and graphs. The single batching machine at stage 2 is capable of processing a batch as long as the batches' total size is less than or equal to its capacity.

Some basic assumptions about the presented problem are summarised as follows.

(1) All jobs (i.e. glasses) are available at the beginning.
(2) All machines are available during all periods of time.
(3) Jobs are scheduled non-preemptively on any machine at both stages.

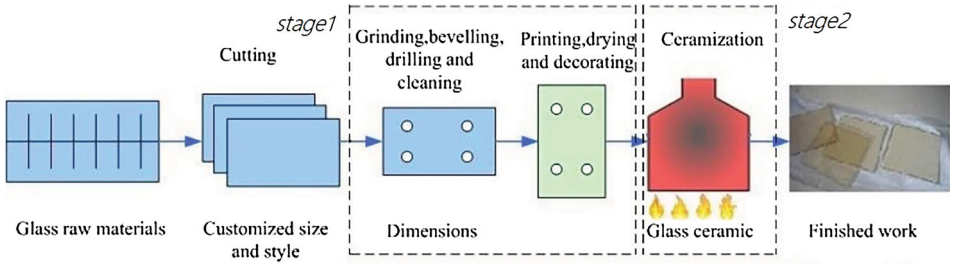

Figure 1. Schematic illustration of the glass-ceramics production process.

(4) Every piece of glass should be processed on exactly one of the parallel machines at stage 1, followed by the heating processing on the single batch machine at stage 2.
(5) Three states of machines are considered: working, idle or turnoff, which implies different energy consumption rate.
(6) More than one piece of glasses could form a batch for heating and ceramisation in the furnace, and the processing time of a batch is determined by the longest processing time of jobs contained in the current batch.
(7) For the parallel machines at stage 1 and the batch machine at stage 2, once a machine is turned off, then it will not be started again during the planned time horizon. In other words, all machines will not be turned off until all the assigned jobs are completed.

In order to minimise the makespan and the energy consumption simultaneously, several decisions should be made: (i) determine the assignment of jobs on machines at stage 1, (ii) determine the starting time of each job at stage 1, (iii) form batches such that the size of furnace is not violated, (iv) determine the starting time of each batch at stage 2. Using the well-known three-field notation, the problem can be denoted by $\{HFS, TOU\}|B|\{C_{\max}, TEC\}$, where '*HFS*' represents the environment of hybrid flow shop, '*TOU*' represents the time-of-use policy of energy price, 'B' represents the batch processing, '$C_{\max}$' represents the makespan and '*TEC*' represents the total energy consumption.

3.2. Model formulation

In this section, a mixed integer programming (MIP) is formulated. Firstly, the parameters are summarised and the decision variables are defined.

Indices:

j: index of job, $j \in \{1, \ldots, n\}$; where n is the total number of jobs.
i: index of machine at stage 1, $i \in \{1, \ldots, m\}$; where m is the total number of machines.
t: index of time period, $t \in \{1, \ldots, T\}$; where T is the total number of time periods. Each time period t begins at time point $(t-1)$ and ends at time point t, i.e. time interval $[t-1, t)$. For instance, time period $t=1$ refers to the time interval $[0, 1)$, $t=2$ refers to the time interval $[1, 2)$, and so on.
b: index of batch, $b \in \{1, \ldots, B\}$; where B is the total number of batches, originally, $B=n$.

Parameters:

$\mathcal{N}$: set of jobs, i.e. $\mathcal{N} = \{1, \ldots, n\}$.
$\mathcal{M}$: set of machines, i.e. $\mathcal{M} = \{1, \ldots, m\}$.
$\mathcal{T}$: set of time periods, i.e. $\mathcal{T} = \{1, \ldots, T\}$.
$\mathcal{B}$: set of batches, i.e. $\mathcal{B} = \{1, \ldots, B\}$.
α_{ij}: 1 if job j can be processed on machine i at stage 1, 0 otherwise.
p_j^1: processing time of job j at stage 1.
p_j^2: processing time of job j at stage 2.
c_t: unit energy price for time period $t \in \mathcal{T}$.
q_j: size of job j.
Q: capacity of the batching machine.
e_i^1: the energy consumption rate of machine i in working.
e_i^2: the energy consumption rate of machine i in idle.
e_i^3: the energy consumption rate of machine i in turnoff (e_i^3 is always a zero vector).
E_1: the energy consumption rate of the batch machine in working.
E_2: the energy consumption rate of the batch machine in idle.
E_3: the energy consumption rate of the batch machine in turnoff (E_3 always equals to zero).

Decision variables:

x_{ij}^t: 1 if job j is processed on machine i at period t, 0 otherwise.
z_{ij}: 1 if job j is assigned on machine i, 0 otherwise.
y_{it}^1: 1 if machine i is working at period t, 0 otherwise.
y_{it}^2: 1 if machine i is idle at period t, 0 otherwise.
y_{it}^3: 1 if machine i is turned off at period t, 0 otherwise.
C_j: the completion time of job j at stage 1.
u_{jb}: 1 if job j is assigned to batch b, 0 otherwise.
v_b: 1 if batch b is formed with at least one job, 0 otherwise.

w_{bt}: 1 if batch b is processing at period t, 0 otherwise.
h_t^1: 1 if the batch machine is processing a batch at period t, 0 otherwise.
h_t^2: 1 if the batch machine is idle at period t, 0 otherwise.
h_t^3: 1 if the batch machine is turned off at period t, 0 otherwise.
P_b: the processing time of batch b.
R_b: the starting time of batch b at stage 2.
$C_{\max}$: the makespan.
β_{it}^1, β_t^2: two auxiliary binary variables.

Then the mathematical formulation for the problem is as follows:

[MIP]

$$\text{minimise} \quad f_1 = C_{\max} \tag{3.1a}$$

$$\text{minimise} \quad f_2 = \sum_{t\in\mathcal{T}}\sum_{i\in\mathcal{M}} c_t(y_{it}^1 e_i^1 + y_{it}^2 e_i^2 + y_{it}^3 e_i^3) + \sum_{t\in\mathcal{T}} c_t(h_t^1 E_1 + h_t^2 E_2 + h_t^3 E_3) \tag{3.1b}$$

$$\text{subject to} \quad \sum_{i\in\mathcal{M}} z_{ij} = 1, \quad \forall j \in \mathcal{N}; \tag{3.1c}$$

$$z_{ij} \le \alpha_{ij}, \quad \forall j \in \mathcal{N},\ i \in \mathcal{M}; \tag{3.1d}$$

$$y_{it}^1 = \sum_{j\in\mathcal{N}} x_{ij}^t, \quad \forall i \in \mathcal{M},\ t \in \mathcal{T}; \tag{3.1e}$$

$$\sum_{i\in\mathcal{M}}\sum_{t\in\mathcal{T}} x_{ij}^t = p_j^1, \quad \forall j \in \mathcal{N}; \tag{3.1f}$$

$$z_{ij} \ge x_{ij}^t, \quad \forall j \in \mathcal{N},\ i \in \mathcal{M},\ t \in \mathcal{T}; \tag{3.1g}$$

$$z_{ij} \le \sum_{t\in\mathcal{T}} x_{ij}^t, \quad \forall j \in \mathcal{N},\ i \in \mathcal{M}; \tag{3.1h}$$

$$(1 - \sum_{i\in\mathcal{M}} x_{ij}^t)T + t \ge C_j - p_j^1, \quad \forall j \in \mathcal{N},\ t \in \mathcal{T}; \tag{3.1i}$$

$$(1 - \sum_{i\in\mathcal{M}} x_{ij}^t)T + C_j \ge t + 1, \quad \forall j \in \mathcal{N},\ t \in \mathcal{T}; \tag{3.1j}$$

$$y_{it}^1 + y_{it}^2 + y_{it}^3 = 1, \quad \forall i \in \mathcal{M},\ t \in \mathcal{T}; \tag{3.1k}$$

$$(1 - y_{it}^2)T + \sum_{t'=1}^{t-1} y_{it'}^1 \ge 1, \quad \forall i \in \mathcal{M},\ t \in \mathcal{T}\backslash\{1\}; \tag{3.1l}$$

$$(1 - y_{it}^2)T + \sum_{t'=t+1}^{T} y_{it'}^1 \ge 1, \quad \forall i \in \mathcal{M},\ t \in \mathcal{T}\backslash\{T\}; \tag{3.1m}$$

$$(1 - y_{it}^3 + \beta_{it}^1)T \ge \sum_{t'=1}^{t-1} y_{it'}^1, \quad \forall i \in \mathcal{M},\ t \in \mathcal{T}\backslash\{1\}; \tag{3.1n}$$

$$(2 - y_{it}^3 - \beta_{it}^1)T \ge \sum_{t'=t+1}^{T} y_{it'}^1, \quad \forall i \in \mathcal{M},\ t \in \mathcal{T}\backslash\{T\}; \tag{3.1o}$$

$$y_{i1}^2 = 0, \quad \forall i \in \mathcal{M}; \tag{3.1p}$$

$$y_{iT}^2 = 0, \quad \forall i \in \mathcal{M}; \tag{3.1q}$$

$$\sum_{b\in\mathcal{B}} u_{jb} = 1, \quad \forall j \in \mathcal{N}; \tag{3.1r}$$

$$\sum_{j\in\mathcal{N}} q_j u_{jb} \le Q v_b, \quad \forall b \in \mathcal{B}; \tag{3.1s}$$

$$\sum_{b\in\mathcal{B}} w_{bt} = h_t^1, \quad \forall t \in \mathcal{T}; \tag{3.1t}$$

$$P_b \geq p_j^2 u_{jb}, \quad \forall j \in \mathcal{N},\ b \in \mathcal{B}; \tag{3.1u}$$

$$\sum_{t\in\mathcal{T}} w_{bt} = P_b, \quad \forall b \in \mathcal{B}; \tag{3.1v}$$

$$v_b T \geq P_b, \quad \forall b \in \mathcal{B}; \tag{3.1w}$$

$$v_b \geq v_{b+1}, \quad \forall b \in \mathcal{B}\backslash\{B\}; \tag{3.1x}$$

$$v_b \geq u_{jb}, \quad \forall j \in \mathcal{N}, b \in \mathcal{B}; \tag{3.1y}$$

$$v_b \leq \sum_{j\in\mathcal{N}} u_{jb}, \quad \forall b \in \mathcal{B}; \tag{3.1z}$$

$$(1-u_{jb})T + R_b \geq C_j, \quad \forall j \in \mathcal{N}, b \in \mathcal{B}; \tag{3.1aa}$$

$$R_b + P_b \leq R_{b+1}, \quad \forall b \in \mathcal{B}\backslash\{B\}; \tag{3.1ab}$$

$$(1-w_{bt})T + t \geq R_b, \quad \forall b \in \mathcal{B},\ t \in \mathcal{T}; \tag{3.1ac}$$

$$(1-w_{bt})T + R_b + P_b \geq t+1, \quad \forall b \in \mathcal{B},\ t \in \mathcal{T}; \tag{3.1ad}$$

$$h_t^1 + h_t^2 + h_t^3 = 1, \quad \forall t \in \mathcal{T}; \tag{3.1ae}$$

$$(1-h_t^2)T + \sum_{t'=1}^{t-1} h_{t'}^1 \geq 1, \quad \forall t \in \mathcal{T}\backslash\{1\}; \tag{3.1af}$$

$$(1-h_t^2)T + \sum_{t'=t+1}^{T} h_{t'}^1 \geq 1, \quad \forall t \in \mathcal{T}\backslash\{T\}; \tag{3.1ag}$$

$$(1-h_t^3+\beta_t^2)T \geq \sum_{t'=1}^{t-1} h_{t'}^1, \quad \forall t \in \mathcal{T}\backslash\{1\}; \tag{3.1ah}$$

$$(2-h_t^3-\beta_t^2)T \geq \sum_{t'=t+1}^{T} h_{t'}^1, \quad \forall t \in \mathcal{T}\backslash\{T\}; \tag{3.1ai}$$

$$(R_b + P_b) + T(v_b - 1) \leq C_{\max}, \quad \forall b \in \mathcal{B}; \tag{3.1aj}$$

$$C_{\max} \leq T, \tag{3.1ak}$$

$$x_{ij}^t, z_{ij}, y_{it}^1, y_{it}^2, y_{it}^3, h_t^1, h_t^2, h_t^3, \beta_{it}^1, \beta_t^2 \in \{0,1\}, \quad \forall j \in \mathcal{N},\ i \in \mathcal{M},\ t \in \mathcal{T}; \tag{3.1al}$$

$$u_{jb}, v_b, w_{bt} \in \{0,1\}, \quad \forall j \in \mathcal{N},\ t \in \mathcal{T},\ b \in \mathcal{B}; \tag{3.1am}$$

$$C_j, R_b, P_b, C_{\max} \geq 0, \quad \forall j \in \mathcal{N},\ b \in \mathcal{B}; \tag{3.1an}$$

The two objectives: (3.1a) minimises the makespan and (3.1b) minimises the total energy consumption (*TEC*) for processing all jobs. Constraint (3.1c) ensures that each job can only be processed on exactly one machine. Constraint (3.1d) indicates the machine eligibility. If job j can not be processed on machine i (i.e. $\alpha_{ij} = 0$), then the assignment variable z_{ij} should take zero. Constraint (3.1e) defines the working state of machine i at period t at stage 1. Constraint (3.1f) ensures that the processing time of job j at stage 1 is equal to p_j^1. Constraints (3.1g) and (3.1h) state the relationship between variables x_{ij}^t and z_{ij}. More specifically, if job j is not assigned to machine i, i.e. $z_{ij} = 0$, then the job will not be processed on this machine at any time period t, i.e. $x_{ij}^t = 0$, $\forall t \in \mathcal{T}$. If job j is assigned to machine i, then we have $z_{ij} = 1 \leq \sum_{t\in\mathcal{T}} x_{ij}^t$, which ensures that there must be at least one time period t that machine i is processing job j. Constraints (3.1i) and (3.1j) ensure that the processing of jobs cannot be interrupted, i.e. jobs have to be scheduled non-preemptively on machines. Constraint (3.1k) guarantees that at certain period t, machine i can be one of three states: working, idle or turned off. Constraints (3.1l) and (3.1m) define the idle state of machines at stage 1. Specifically, these two constraints make sure that if machine i is idle at time period t, then it must have worked at least one period during periods from 1 to $(t-1)$ and it will work at least one period during periods from $(t+1)$ to T (otherwise, it would be turned off). Constraints (3.1n) and (3.1o) define the turnoff state of machines at stage 1. More specifically, they coordinate that if machine i is turned off at time

period t, then it cannot process any job during periods from $(t+1)$ to T, i.e. the machine will not be restarted once it is turned off, which implies $\sum_{t'=t+1}^{T} y_{it'}^{1} = 0$. And there is no such restrictions when the machine is not in the turnoff state at the time period t. Constraints (3.1p) and (3.1q) require that all of the machines at stage 1 are not idle in the first and the last period T.

Constraints (3.1r)–(3.1t) represent the restrictions of batch forming and batch processing required at stage 2. Constraint (3.1r) ensures that each job is assigned to exactly one batch. Constraint (3.1s) guarantees that the size of a formed batch b is no larger than the capacity of the batch machine. Constraint (3.1t) indicates that there is exactly one batch b being processed at period t. Constraints (3.1u)–(3.1w) describe the processing time of batches. Among them, constraint (3.1u) indicates that the processing time of batch b is the longest processing time of all jobs in this batch. Constraint (3.1v) defines the processing time of batch b at stage 2. Constraint (3.1w) describes such a special case that, if batch b is not formed, i.e. $v_b = 0$, then the processing time of batch b must take the value of zero. Constraints (3.1x)–(3.1z) represent the restriction of forming batch. Constraints (3.1x) exclude the circumstance that batch $b+1$ is formed without forming batch b. Constraints (3.1y) and (3.1z) coordinate the values of v_b and u_{jb}. Constraints (3.1aa)–(3.1ab) define the starting and completion time of batch b. Constraint (3.1aa) guarantees that the starting time of batch b is larger than or equal to the completion time of jobs at Stage 1 in this batch. Constraint (3.1ab) indicates that the starting time of batch $b+1$ is at least equal to the completion time of batch b. Constraints (3.1ac)–(3.1ad) guarantee that the processing of a batch b on the batch machine will not be interrupted. They ensure that if batch b is processed at period t, i.e. $(1 - w_{bt})T = 0$, then the starting time of batch b is smaller than or equal to t, and the completion time of batch b is larger than or equal to $(t+1)$. Constraint (3.1ae) indicates that there is only one state for the batch machine at any period t: working, idle and turned off. Constraints (3.1af) and (3.1ag) define the idle state of the batch machine at stage 2. They deal with the same issue as constraints (3.1l) and (3.1m). Constraints (3.1ah) and (3.1ai) define the turnoff state of the batch machine, similar as constraints (3.1n) and (3.1o). Constraint (3.1aj) defines the makespan and constraint (3.1ak) ensures that the makespan is not larger than the number of total planning time periods. Finally, constraints (3.1al)–(3.1an) give the range of variables.

We take an example to illustrate the model. The augmented ϵ-constraint method (AUGMECON) in Gurobi 7.5 is adopted to obtain the exact Pareto solutions of a randomly generated instance with eight jobs and three machines at stage 1, which is denoted by Ex 1. In Ex 1, the capacity of furnace is $Q=7$. The energy consumption rates (ECR) for different states of machines are given in Table 1. Table 2 presents the processing-related information including machine eligibility, processing time (PT) and size of jobs. In this table, '\' represents that the corresponding job cannot be processed on the corresponding machine. Figure 2 shows the TOU energy prices in detail.

For the adoption of the augmented ϵ-constraint method, we need to obtain the lower bound and upper bound of the objective, i.e. Ideal point $\mathbf{f}^I = (f_1^I, f_2^I)$ and Nadir point $\mathbf{f}^N = (f_1^N, f_2^N)$ (Bérubé, Gendreau, and Potvin 2009). In this problem, the makespan f_1 is used as a constraint, $\Delta = 1$ as the minimum unit of value change of the makespan. *eps* is an small number

Table 1. Energy consumption rates of machines in Ex 1.

Stage	*Machines*	$ECR_{working}$	ECR_{idle}	$ECR_{turnoff}$
1	M_1	4	1	0
	M_2	2	1	0
	M_3	3	1	0
2	*Furnace*	5	2	0

Table 2. Processing features of jobs and machines in Ex 1.

	Stage 1			*Stage* 2	
Jobs	PT_{M_1}	PT_{M_2}	PT_{M_3}	$PT_{Furnace}$	*Size*
1	5	5	5	5	2
2	5	\	5	4	2
3	\	5	5	5	5
4	1	1	\	5	5
5	3	\	3	2	2
6	1	1	1	2	2
7	3	\	3	4	2
8	5	5	\	2	2

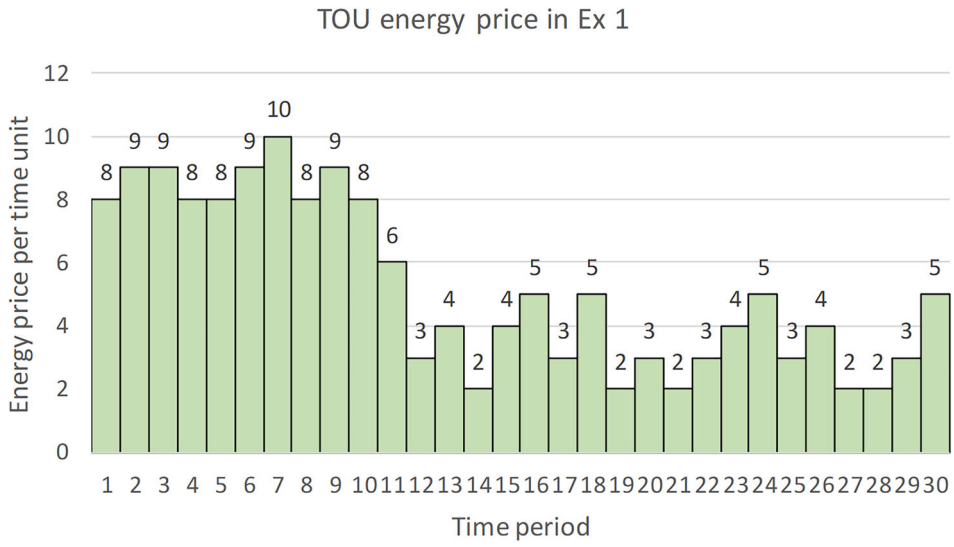

Figure 2. The detailed TOU energy prices in Ex 1.

(usually between 10^{-3} and 10^{-6}, we set $eps = 10^{-4}$), s is a positive integer variable. The detailed procedure of augmented ϵ-constraint method for this problem is given in Algorithm 3.1.

Algorithm 3.1 The augmented ϵ-constraint method

1: Compute the Ideal point $\mathbf{f}^I = (f_1^I, f_2^I)$ and Nadir point $\mathbf{f}^N = (f_1^N, f_2^N)$.
2: Set $\mathbf{F}$={(f_1^N, f_2^I)} and $\epsilon = f_1^N$ - Δ (Δ =1 for this problem).
3: **while** $\epsilon \geq f_1^I$ **do**
4: Solve the ϵ-constraint problem by adding $C_{\max} + s = \epsilon$ as an additional constraint, and the TEC - $eps \times s$ as the single objective function to optimise. Add the optimal solution value (f_1^*, f_2^*) to $\mathbf{F}$.
5: Update $\epsilon = f_1^*$ - Δ, repeat the above steps until $\epsilon \geq f_1^I$ is not satisfied.
6: **end while**
7: Obtain the Pareto front $\mathbf{F}$.

With the Gurobi 7.5 solver, the generated optimal Pareto front contains 13 non-dominated solutions, which are shown in Figure 3. The front illustrates the trade-off relationship between $C_{\max}$ and TEC, which also indicates that the single-objective

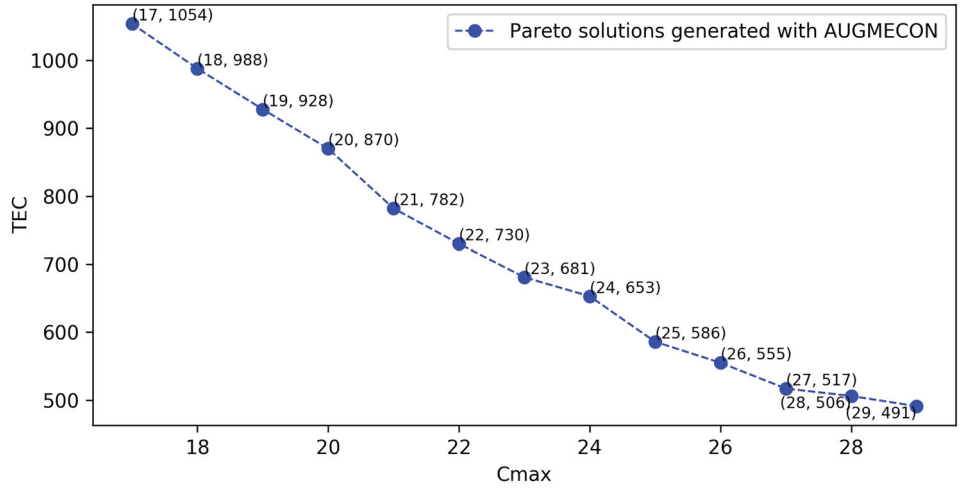

Figure 3. The optimal Pareto front of Ex 1.

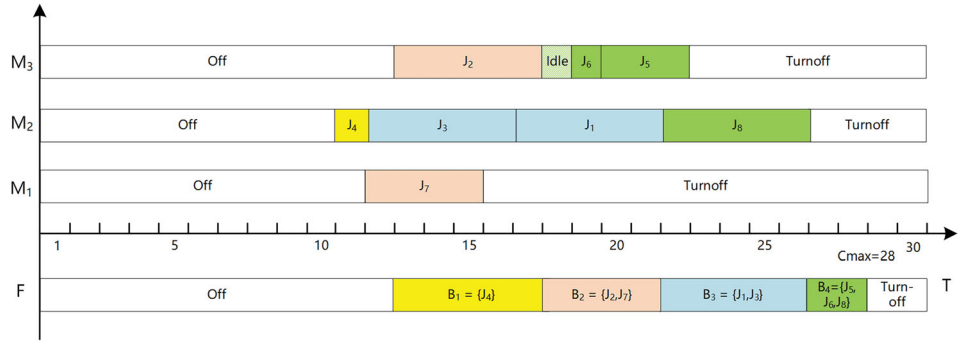

Figure 4. The illustration Gantt chart of solution (28, 506) in Ex 1 (generated with AUGMECON).

of minimising makespan may cost much higher energy consumption and vice versa. More specifically, Figure 4 presents the Gantt charts for the detailed scheduling of solution (28, 506). All jobs have been assigned to their eligible machines, and four batches are formed. In this figure, 'F' represents the furnace for batch processing.

Since the special case of the problem $\{HFS, TOU\}|B|\{C_{\max}, TEC\}$ with $c_t = 0$ for every $t \in \mathcal{T}$ has been proved to be strongly NP-hard (Lee and Vairaktarakis 1994), $\{HFS, TOU\}|B|\{C_{\max}, TEC\}$ is NP-hard in the strong sense. Hence, general purpose commercial solvers can only obtain (optimal) solutions for small-scale problems. Effective heuristic and meta-heuristic algorithms are needed for practical application.

4. Solution approach

In this section, a constructive heuristic method is proposed first, based on which a bi-objective tabu search and a bi-objective ant colony optimisation algorithm are designed to obtain the approximate Pareto front for medium- and large-scale problems.

4.1. A constructive heuristic method with local search strategy

4.1.1. A constructive heuristic procedure

In this subsection, a problem-tailored constructive heuristic with a local search strategy is proposed, which can be divided into three steps.

(1) Step 1: Schedule jobs in parallel machines at stage 1. Three rules are used in this step: (i) longest processing time p^1 (LPT) first, combined with the minimum local energy consumption (MLEC) rule. The LPT rule is used to determine the sequence of jobs to be scheduled. The MLEC rule is adopted to determine the machine assignment and starting time of jobs. The MLEC rule is based on such an idea: when $C_{\max}$ is limited by a certain value, an optimal schedule tries to avoid the job processing on machines with high energy consumption rate and at periods with high energy prices. Consequently, assigning jobs into an eligible machine with minimum energy cost tends to be rather intuitive. (ii) shortest processing time p^1 (SPT) first, combined with MLEC rule, and (iii) shortest processing time p^2 (SPT) first, with earliest completion time (ECT) first.
(2) Step 2: Forming the batches. Two rules are used: (i) first come, first served (FCFS) with full batch rule (FB), and (ii) shortest processing time p^2 (SPT) first with FB rule. Full batch rule means that jobs are assigned into one batch until the capacity of the furnace is exceeded.
(3) Step 3: Schedule batches in the furnace. The MLEC rule as well as ECT rule are both applied in the last step.

The complete procedure of the constructive heuristic is given in Algorithm 4.1. The main idea is to try different rules for assigning jobs at stage 1, forming batches and scheduling batches at Stage 2. If the schedule with a certain rule is failed, i.e. cannot find feasible solution, another rules will be applied. A flow chart is given in Figure 5 to present the logic of Algorithm 4.1 more clearly. To leave sufficient time for batch processing at stage 2, a set of upper bounds of completion time at stage 1, $\mathcal{BD}$, is given. The maximum completion time of jobs at stage 1 cannot be larger than the upper bound, which retains time for the batch processing at stage 2.

Algorithms 4.2, 4.3 and 4.4 describe the detailed process of job scheduling, batch forming and batch scheduling, respectively. Figure 6 illustrates the detailed schedule with the constructive heuristic method for solution (28, 551) in Ex 1. Compared with the solution (28, 506) generated by AUGMECON (see Figure 4), the constructive heuristic obtains a larger $TEC = 551$ but with the same makespan $C_{\max} = 28$.

Algorithm 4.1 The constructive heuristic method

1: Input: the information of jobs (n, p^1, p^2, q), machines $(m, e^1, e^2, E1, E2, Q)$, energy prices c_t, and the set of upper bounds of completion time at stage 1, $\mathcal{BD} = \{\frac{3}{4}T, \frac{1}{2}T, \frac{1}{3}T\}$.
2: Define $T_{\min} = \max\{\sum_{j=1}^{n} p_j^1/m, \max\{p_j^1, j \in \mathcal{N}\}\}$. Let $\mathcal{NS} = \emptyset$ be the set of possible solutions.
3: **for** every bound, bd ($bd \in \mathcal{BD}$) **do**
4: **while** $T \geq T_{\min}$ **do**
5: Execute the **JobScheduleRule** with $k = 1$ (see Algorithm 4.2), obtain the schedule at stage 1, $Schedule_1$, and a binary indicator indicating whether the scheduling at stage 1 is successful or failed, $Flag_1$.
6: **if** $Flag_1 = 1$ **then**
7: Compute TEC_1 and $C_{\max 1}$ according to $Schedule_1$.
8: Execute the **BatchFormingRule** with $l = 1$ (see Algorithm 4.3), obtain the set of batches $\mathcal{B}$, release time of batches $\mathcal{RB}$ and processing time of batches $\mathcal{PB}$.
9: Execute the **BatchScheduleRule** (see Algorithm 4.4), obtain the schedule at stage 2, $Schedule_2$, and a binary indicator indicating whether the batch scheduling at stage 2 is successful or failed, $Flag_2$.
10: **if** $Flag_2 = 0$ **then**
11: Execute the **BatchFormingRule** with $l = 2$ (see Algorithm 4.3), obtain $\mathcal{B}$, $\mathcal{RB}$ and $\mathcal{PB}$.
12: Execute the **BatchScheduleRule** (see Algorithm 4.4), obtain $Schedule2$ and $Flag2$.
13: **else**
14: Compute TEC_2 and $C_{\max 2}$ according to the $Schedule_2$.
15: Obtain current solution: $s = (C_{\max}, TEC)$, where $C_{\max} = C_{\max 2}$, $TEC = TEC_1 + TEC_2$.
16: Update the current solution s to the set $\mathcal{NS}$: $\mathcal{NS} = \mathcal{NS} \cup \{s\}$.
17: **end if**
18: **if** $Flag_2 = 0$ **then**
19: % *if the batch forming and scheduling using BatchFormingRule with* $l = 2$, *is also failed. Try to reschedule the jobs at stage 1 by another rule, JobScheduleRule with* $k = 2$.
20: Execute the **JobScheduleRule** with $k = 2$ (see Algorithm 4.2), obtain $Schedule_1$ and $Flag_1$.
21: **if** $Flag_1 = 1$ **then**
22: % *if the job scheduling at stage 1 using JobScheduleRule with* $k = 2$, *is successful.*
23: Form batches, perform scheduling and update $\mathcal{NS}$ by following the steps in Lines $7 - 17$. If $Flag_2 = 0$ then no solution is obtained.
24: **else**
25: No solution obtained, continue.
26: **end if**
27: **else**
28: Obtain the current solution and update $\mathcal{NS}$ by following the steps in Lines $14 - 16$.
29: **end if**
30: **else**
31: Execute the **JobScheduleRule** with $k = 3$ (see Algorithm 4.2), obtain the $Schedule_1$ and $Flag_1$.
32: **if** $Flag_1 = 1$ **then**
33: % *if the job scheduling at stage 1 using JobScheduleRule with* $k = 3$, *is successful.*
34: Form and schedule batches, then update $\mathcal{NS}$ by following the steps in Lines $7 - 17$. If $Flag_2 = 0$ then no solution is obtained.
35: **else**
36: No solution obtained, continue.
37: **end if**
38: **end if**
39: Update the time periods: $T = C_{\max} - 1$, where '$C_{\max}$' represents the minimal makespan of all the obtained solutions for now.
40: **end while**
41: **end for**
42: Improve the solutions in set $\mathcal{NS}$ with proposed **local search strategy** (see Algorithm 4.5).
43: Delete the dominated solutions in set $\mathcal{NS}$, if any.
44: Output: the set of non-dominated solutions, $\mathcal{NS}$.

4.1.2. The local search strategy

The basic idea of the local search strategy mentioned in Line 42 in Algorithm 4.1 is described as follows. If some jobs are processing on the same machine consecutively, i.e. without idle or turnoff time periods, then these jobs could be handled as one single job block. Consequently, the change of the start time of a job block may bring some reduction of energy

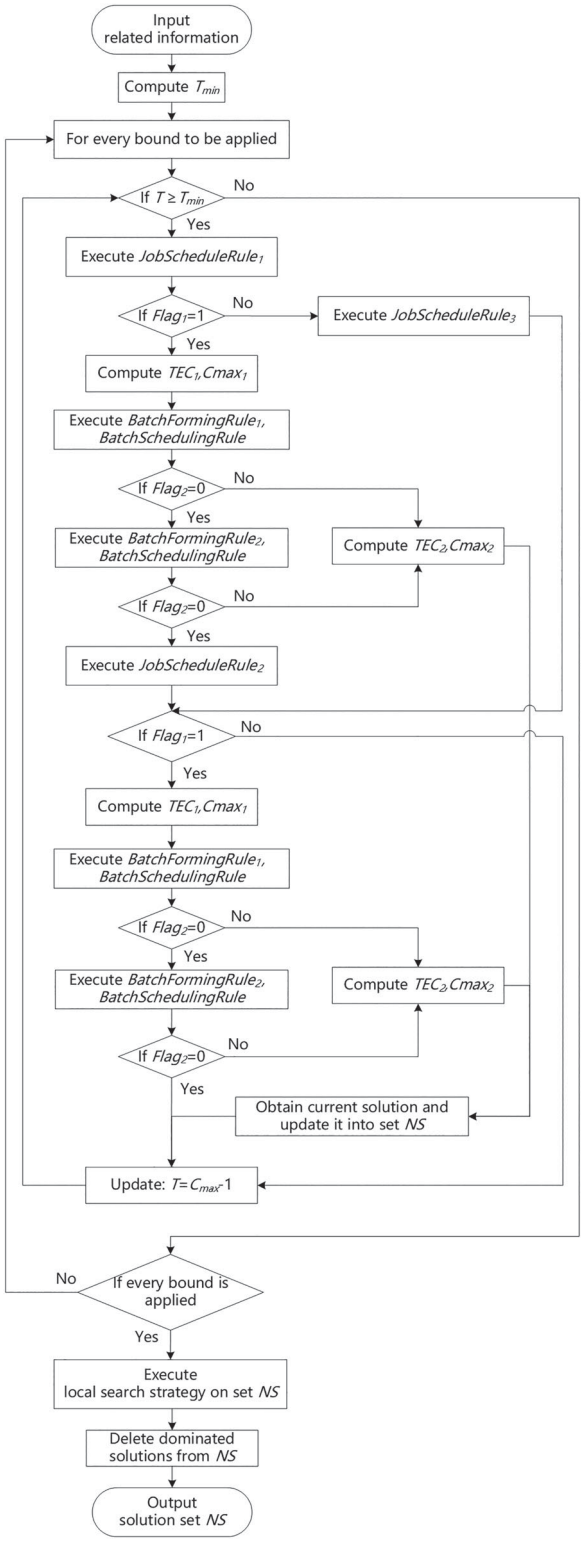

Figure 5. The flow chart of the constructive heuristic method for Algorithm 4.1

consumption, due to the TOU prices. For instance, Figure 7 represents the improvement of solution (28, 551) in Figure 6 with proposed local search strategy. More specifically, the starting time of job block $j = 8$ is delayed from $ST_8^1 = 17$ to $ST_8^1 = 19$ on machine $i = 3$ at stage 1. The completion time of this job block now is $CT_8^1 = 23$, which is still earlier than $ST_4^2 = 27$, the starting time of batch $b = 4$ that consists of job $j = 8$. Consequently, the $C_{\max}$ does not change, while the *TEC* decreases

Algorithm 4.2 The pseudo-code of JobScheduleRule k

Input: the information of jobs (n, p^1, p^2), machines (m, e^1, e^2), energy prices c_t, time horizon T, upper bound of completion time at stage 1, $\mathcal{BD}_{bd}$, and the JobScheduleRule index k, where $k \in \{1, 2, 3\}$.
Define $h^1_{it} = e^1_i c_t$ and $h^2_{it} = e^2_i c_t$, where $i = \{1, \ldots, m\}, t = \{1, \ldots, T\}$.
if $k \neq 3$ **then**
  if $k = 1$ **then**
    % *sort the jobs with LPT rule.*
    Sort the jobs in non-increasing order of processing times at stage 1, $p^1_j, j \in \mathcal{N}$, i.e., $p^1_{[1]} \geq p^1_{[2]} \geq \cdots \geq p^1_{[n]}$, where $[j]$ is the jth job in the order.
  else if $k = 2$ **then**
    % *sort the jobs with SPT rule.*
    Sort the jobs in non-decreasing order of processing times at stage 2, $p^2_j, j \in \mathcal{N}$, i.e., $p^2_{[1]} \leq p^2_{[2]} \leq \cdots \leq p^2_{[n]}$, where $[j]$ is the jth job in the order.
  end if
  Initialize $TECP = 0$.
  for every job, j ($j \in \mathcal{N}$) **do**
    % *schedule the jobs with MLEC rule.*
    Search certain time interval with minimum $TECP = TECP + \Delta TECP^j_{it}$ from all available and continuous time periods on every eligible machine $i = \{1, \ldots, m\}$, with $t = \{1, \ldots, \min\{T - (p^1_j - 1), \mathcal{BD}_{bd}\}\}$, where $\Delta TECP^j_{it}$ represents the variation of total energy consumption (including working and idle states) after current job j is assigned to machine i and processed from time period t, and $TECP$ denotes the total energy consumption of partial schedule for jobs $[1] - [j]$. This selected time interval on machine i^*_j is the best option for job j since it causes the least energy consumption for processing job j. The earlier starting time is used as the tie-breaking for the same minimum $TECP$.
    Update the detailed schedule of current job in $Schedule_1$.
  end for
  if n jobs are scheduled successfully on m machines within BD_{bd} time periods **then**
    $Flag_1 = 1$.
  else
    $Flag_1 = 0$.
  end if
else
  Sort the jobs with SPT rule by follow the step in Line 9.
  Schedule jobs by following the ECT rule.
  Obtain the indicator $Flag_1$ by following the steps in Lines $17 - 21$.
end if
Output: the indicator $Flag_1$, and job scheduling at stage 1 $Schedule_1$.

by $e^1_3(c_{17} + c_{18}) + e^2_3(c_{22} + c_{23}) - e^2_3(c_{17} + c_{18}) - e^1_3(c_{22} + c_{23}) = 3(3 + 5) + 1(3 + 4) - 1(3 + 5) - 3(3 + 4) = 2$, since partial job block is being processed at time periods from $t = 22$ to 23 with lower energy prices $c_t = \{3, 4\}$, rather than from $t = 17$ to 18 ($c_t = \{3, 5\}$). Note that the main idea of the local search algorithm is similar as that of the right-shift electric power cost (EPC) reduction procedure in Luo et al. (2013). However, the adoption of moving job blocks rather than single jobs only provides potential to find more promising solutions. The detailed local search procedure is given in Algorithm 4.5.

4.2. A bi-objective tabu search algorithm

In this subsection, a bi-objective tabu search algorithm integrated with the proposed constructive heuristic method is developed to obtain solutions for practical-size problems in reasonable computation time.

Tabu search was first proposed by Glover (1986), in which the well-known tabu list is introduced with the consideration of avoiding the local optima. When a local optimum is detected, current solution or related movement that generated this optimum will be updated into the tabu list. The elements in the tabu list will be forbidden for a certain number of iterations.

4.2.1. The representation of solutions

The representation of solution is a matrix with $m + 1$ rows and T columns, where m is the number of parallel machines at stage 1, and T is the number of time periods in the planned horizon. The elements in the first m rows (machines) are coded with values from the set $\{1, 2, \ldots, n\}$, representing which job j ($j \in \mathcal{N}$) is processed on current machine i ($i \in \mathcal{M}$). The

Algorithm 4.3 The pseudo-code of BatchFormingRule l

Input: the information about jobs (p^2, q), furnace ($E1, E2, Q$), energy prices c_t, time horizon T, the scheduling at stage 1 *Schedule*1, and the BatchFormingRule index l, where $l \in \{1, 2\}$.
Let the set of batches, their release time and processing time be $\mathcal{B} = \emptyset$, $\mathcal{RB} = \emptyset$, and $\mathcal{PB} = \emptyset$, respectively.
Define $H_t^1 = E_1 c_t$ and $H_t^2 = E_2 c_t$, where $t = \{1, \ldots, T\}$.
if $l = 1$ **then**
    % *sort the jobs with FCFS rule.*
    Sort the jobs in non-decreasing order of completion times at stage 1, $C_{\max 1j}$, $j \in \mathcal{N}$, i.e., $C_{\max 1[1]} \leq C_{\max 1[2]} \leq \cdots \leq C_{\max 1[n]}$, where $[j]$ is the jth job in the order.
end if
if $l = 2$ **then**
    % *sort the jobs with SPT rule.*
    Sort the jobs in non-decreasing order of processing times at stage 2, p_j^2, $j \in \mathcal{N}$, i.e., $p_{[1]}^2 \leq p_{[2]}^2 \leq \cdots \leq p_{[n]}^2$, where $[j]$ is the jth job in the order.
end if
Let the size of current batch be $cq = 0$.
for every job, j ($j \in \mathcal{N}$) **do**
    % *form batches with FB rule.*
    if $cq + q_j \leq Q$ **then**
        Job j is assigned to current batch, and update $cq = cq + q_j$.
        Update the detailed assignment of current job in $\mathcal{B}$.
    else
        Start a new batch and reset $cq = 0$.
        Job j is assigned to this new batch, and update $cq = cq + q_j$.
        Update the detailed assignment of current job in $\mathcal{B}$.
    end if
end for
for $b = 1 : |\mathcal{B}|$ **do**
    $\mathcal{RB}_b = \max_{j \in \mathcal{B}_b}\{C_{\max 1j}\}$.
    $\mathcal{PB}_b = \max_{j \in \mathcal{B}_b}\{p_j^2\}$.
end for
Output: the set of batches $\mathcal{B}$, release time of batches $\mathcal{RB}$ and processing time of batches $\mathcal{PB}$.

starting time of job j at stage 1 is determined by the position of the first number 'j'. For instance, if the processing time of job 1, $p_1^1 = 3$, and number '$j = 1$' occurred on the first, second and third positions of the ith row in the matrix, then the starting time of job 1 in machine i is $ST_1^1 = 1$. Similarly, the elements in the $(m + 1)$th row are coded with a value from the set $\{1, 2, \ldots, B\}$, where B is the number of formed batches in this solution. And the starting time of batch b ($b \in \mathcal{B}$) is determined with the same idea.

4.2.2. Initialisation

The initialisation is designed to generate an initial solution for the tabu search procedure. The proposed constructive heuristic method in Algorithm 4.1 is embedded into the initialisation procedure and generates a set of non-dominated solutions. After that, one of them will be selected randomly as the initial solution to obtain neighbour solutions during the tabu search process.

4.2.3. The neighbourhood structure

Considering the bi-objective characteristic of the problem, we design a novel neighbourhood structure to find the neighbour solutions. The main idea is to decrease the maximum $C_{\max 1}$ of the m parallel machines at stage 1, as well as the $C_{\max 2}$ in the furnace. More specifically, for the incumbent solution, the machine i^* with maximum completion time at stage 1 is selected.

Then the job schedule on machine i^* will be adjusted as follows: the job blocks are found, and the algorithm will try to move each block towards left by a randomly generated amount of time periods, which may result in a smaller $C_{\max 1}$ as well as the reduction of energy consumption at stage 1. Meanwhile, the batch scheduling at stage 2 also aims to decrease the $C_{\max 2}$ by using the *FCFS* and full batch rule. A small amount of delay randomly generated is integrated at stage 2, which

Algorithm 4.4 The pseudo-code of BatchScheduleRule

1: Input: the information about the furnace $(E1, E2)$, energy prices c_t, time horizon T, the set of batches $\mathcal{B}$, the set of batches' release time $\mathcal{RB}$ and the set of batches' processing time $\mathcal{PB}$.
2: Define $H_t^1 = E_1 c_t$ and $H_t^2 = E_2 c_t$, where $t = \{1, \ldots, T\}$.
3: Sort the batches in non-decreasing order of release times, $\mathcal{RB}_b$, $b \in \mathcal{B}$, i.e., $\mathcal{RB}_{[1]} \leq \mathcal{RB}_{[2]} \leq \cdots \leq \mathcal{RB}_{[|\mathcal{B}|]}$, where $[b]$ is the bth batch in the order.
4: Initialise the release time of furnace R_f: $R_f = 0$.
5: Initialise $TECP = 0$.
6: **for** every batch, b $(b \in \mathcal{B})$ **do**
7: % *schedule batches with MLEC rule.*
8: The starting time of batch b: $ST_b = \max\{\mathcal{RB}_b, R_f\}$
9: For batch b, search certain time interval with minimum $TECP = TECP + \Delta TECP_t^b$ from all available and continuous time periods on the furnace, with $t = \{ST_b, \ldots, T - (\mathcal{PB}_b - 1)\}$, where $\Delta TECP_t^b$ represents the variation of total energy consumption (including working and idle states) after batch b is processed from time period t, and $TECP$ denotes the total energy consumption of current partial schedule for batches $[1] - [b]$ on the furnace. This selected time interval starting from t_b^* is the best option for batch b since it causes the least working and idle energy consumption. The earlier starting time is used as the tie-breaking for the same minimum $TECP$.
10: Update the working, idle and turnoff states of the furnace.
11: Construct $Schedule_2$ with detailed schedule of current batch b.
12: $R_f = t_b^* + \mathcal{PB}_b - 1$
13: **end for**
14: **if** $|\mathcal{B}|$ batches are scheduled successfully on the furnace within T time periods **then**
15: $Flag_2 = 1$.
16: **else**
17: $Flag_2 = 0$.
18: % *schedule batches with ECT rule.*
19: Construct $Schedule_2$ by following the ECT rule.
20: Update the value of $Flag_2$ by following Lines $14 - 17$.
21: **end if**
22: Output: the indicator $Flag_2$, and batch scheduling at stage 2 $Schedule_2$.

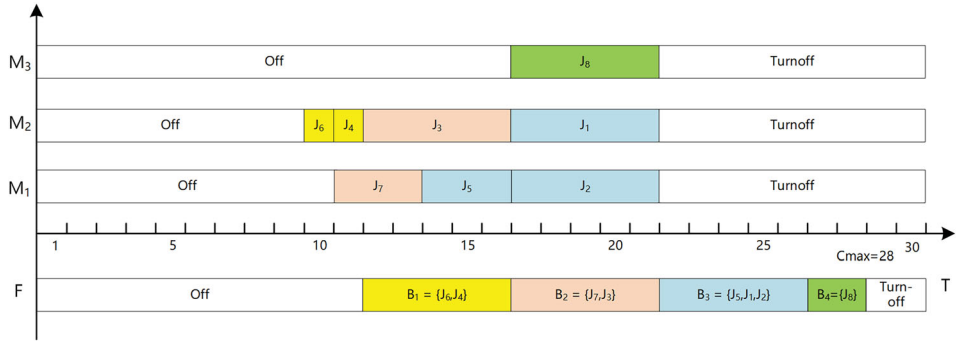

Figure 6. The illustration Gantt chart of solution $(28, 551)$ in Ex 1 (generated with the constructive heuristic).

strengthens the randomness of searching procedure. In each iteration, N_{cd} neighbour solutions are generated, and half of them are retained as the candidate solutions. The detailed procedure is described in Algorithm 4.6.

4.2.4. Tabu list and aspiration criterion

The tabu list is set up to preserve several pairs of two objectives $(C_{\max}, TEC)$ of the local optimums. In other words, the newly generated solutions with objectives remained in the tabu list cannot be visited by the algorithm. While if they are not forbidden by the tabu list, then they will be evaluated. The tabu length L_{tb} is predetermined as a constant, which is also a parameter of the algorithm. Moreover, a simple aspiration criterion is applied. It allows that the generated solutions can be performed if the incumbent solution is non-dominated to all of the current Pareto solutions.

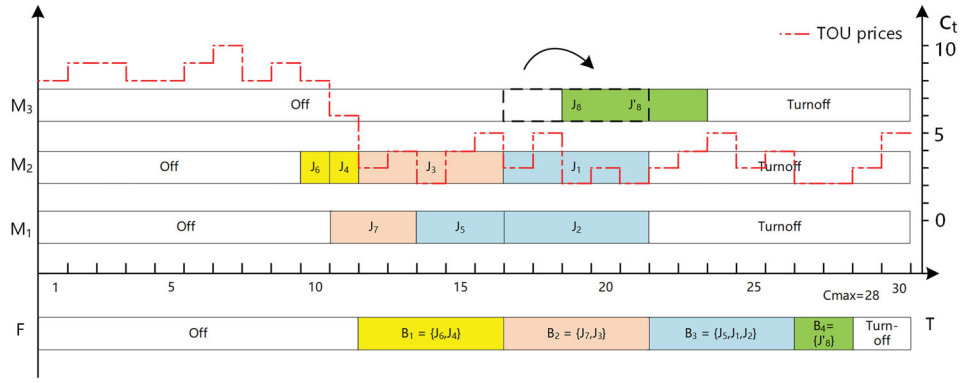

Figure 7. The illustration of local search strategy based on solution in Figure 6.

Algorithm 4.5 The local search procedure

1: Input: The scheduling of two stages: $Schedule_1$, $Schedule_2$, energy price c_t and time horizon T.
2: % *the improvement at stage 1.*
3: **for** every machine, i ($i \in \mathcal{M}$) **do**
4: With $Schedule_1$, find the jobs on machine i which could be seen as one job block. Use the set $\mathcal{JB}_1$, $\mathcal{JB}_2$,…,$\mathcal{JB}_S$ to record them, where S is the total number of all job blocks.
5: **for** every block, jb ($jb \in \{1, 2, \ldots, S\}$) **do**
6: Search the possible change of the starting time for $\mathcal{JB}_{jb}$ which could bring the most reduction of total energy consumption on machine i.
7: Update the state of the machine i and the scheduling in $Schedule_1$.
8: **end for**
9: **end for**
10: % *the improvement at stage 2.*
11: With $Schedule_2$, find the batches on the furnace that could be seen as one job block and donate them as $\mathcal{JB}'_1$, $\mathcal{JB}'_2$,…,$\mathcal{JB}'_{S'}$.
12: Repeat the searching procedure described in Lines 5 − 8 but with a further restriction that the starting time of batch jb, ST_{jb}, is larger than or equal to the completion time of all the jobs assigned to this batch: $ST_{jb} \geq \max_{j \in \mathcal{JB}'_{jb}}\{C_{\max 1 j}\}$.
13: Update the state of periods in the furnace and the scheduling in $Schedule_2$.
14: Output: The updated $Schedule_1$ and $Schedule_2$.

4.2.5. Termination condition

The bi-objective tabu search is terminated after a fixed number of iterations, $I_{\max}$.

4.2.6. The complete bi-objective tabu search algorithm

The proposed bi-objective tabu search framework is described in Algorithm 4.7. Lines 1–2 input parameters and initialise the algorithm. An initial solution is obtained in Lines 3–4. Lines 5–40 are the main loop of tabu search. Neighbour solutions are generated in Line 6 and candidate solutions are selected in Line 7, then the aspiration criterion is tested in every candidate solutions in Lines 9–24. If all the candidate solutions cannot meet the aspiration criterion, the tabu list will be updated in Lines 25–37. During each iteration, the set of candidate solutions is updated into the set $\mathcal{NS}$ in Line 39. After $I_{\max}$ iterations, the algorithm deletes the dominated solutions of $\mathcal{NS}$ in Line 41, and outputs the result in Line 42.

4.3. Bi-objective ant colony optimisation algorithm

In this subsection, the bi-objective ant colony optimisation algorithm is adapted to tackle the large-scale instances. Ant Colony Optimisation (ACO) algorithm is a meta-heuristic proposed by Dorigo (1992) for solving combinatorial optimisation problems. It is inspired by the trail laying and following behaviour of ants, in which certain chemical substances (e.g. pheromones) are used for communication. Based on the indirect information exchanging of ants within a colony, every ant will experience a route from the source to the destination in each iteration. A piece of route, constituted by several nodes, represents one solution. During the procedure of ACO, routes will be constructed probabilistically and

Algorithm 4.6 The GetNeighbor procedure

Input: the initial solution *currentSol*.
According to the schedule in *currentSol*, select machines with maximum completion time at stage 1. Let $\mathcal{M}_{mct}$ be the set of these machines, and T represents the planned time horizon.
for every machine, i^* ($i^* \in \mathcal{M}_{mct}$) **do**
  Let $T^{i^*}_{\max}$ be the makespan on machine i^*. Let $\mathcal{JB}$ be the set of all job blocks (i.e., there are no idle or turnoff during these jobs processing).
  for every job block, jb ($jb \in \mathcal{JB}$) **do**
    Let ST^1_{jb} and CT^1_{jb} be the starting and completion time of block jb at stage 1.
    Sort the job blocks of $\mathcal{JB}$ in a non-decreasing order of ST^1_{jb}.
    if $jb = [1]$ **then**
      $T_{avai} = ST^1_{jb} - 1$.
      Generate $avait_{jb}$ randomly from $[0, T_{avai}]$.
      The starting time of block jb: $ST^1_{jb} = avait_{jb}$.
    else
      $T_{avai} = ST^1_{jb} - CT^1_{jb-1}$.
      Generate $avait_{jb}$ randomly from $[0, T_{avai}]$.
      The starting time of block jb: $ST^1_{jb} = CT^1_{jb-1} + avait_{jb}$.
    end if
    Update the starting time information for block jb in *currentSol*.
  end for
end for
Form batches by following the *BatchFormingRule* with $l = 1$ (see Algorithm 4.3).
Let $\mathcal{B}$, $\mathcal{RB}$ and $\mathcal{PB}$ be the set of formed batches, batches' release time and processing time, respectively.
Initialise the release time of furnace R_f: $R_f = 0$.
for every batch, b ($b \in \mathcal{B}$) **do**
  Generate *prob* randomly from $[0, 1]$.
  if $prob \leq 0.5$ **then**
    The starting time of batch b at stage 2: $ST^2_b = \max\{\mathcal{RB}_b, R_f\}$.
  else
    Generate t_{delay} randomly from $[0, \lceil T - \min_{b\in\mathcal{B}}\{\mathcal{RB}_b\} + 1 - \sum_{b\in\mathcal{B}} \mathcal{PB}_b/|\mathcal{B}|/3\rceil]$.
    The starting time of batch b: $ST^2_b = \max\{\mathcal{RB}_b, R_f\} + t_{delay}$.
  end if
  $R_f = ST^2_b + \mathcal{PB}_b$.
  Update the starting time information for batch b in *currentSol*.
end for
neighborSol = *currentSol*.
Output: the generated neighbour solution, *neighborSol*.

converge to better ones. The detailed implementation of bi-objective ACO for this problem are presented in the following subsections.

4.3.1. The definition of nodes

Considering the characteristics of the problem, a complete route represents a solution. It contains the machine assignment and starting time of jobs at stage 1, the batch forming information and every batch's starting time at stage 2. The nodes that compose a route are defined as: for job j, the starting time ST^1_j on a certain machine i at stage 1 (denoted as node (j, i, ST^1_j)), and the starting time ST^2_b in the furnace for batch b at stage 2 (denoted as node (b, ST^2_b)). Figure 8 illustrates the nodes of two stages for solution $(28, 506)$ presented in Figure 4.

4.3.2. The definition and updating of pheromone

Pheromone is used to search for good nodes at stage 1 during the route construction procedure, which plays a vital role in bi-objective ACO. Note that for different solutions, the batches formed at stage 2 may contain different jobs, which increases the variety of nodes. When bi-objective ACO constructs a route at stage 2, the node searching procedure is guided by *BatchFormingRule* with $l = 2$ in Algorithm 4.3 and *BatchScheduleRule* in Algorithm 4.4.

Algorithm 4.7 The bi-objective tabu search procedure

1: Input: the information of jobs, machines, and energy prices. The maximum tabu length L_{tb}, the maximum number of neighbour solutions N_{cd}, and the maximum number of iterations $I_{\max}$.
2: Let $\mathcal{TB} = \emptyset$ and $\mathcal{NS} = \emptyset$ be the tabu list and the set of solutions, respectively.
3: Generate solutions with the proposed **constructive heuristic method** in Algorithm 4.1. Let $\mathcal{PS}$ record all the obtained Pareto solutions.
4: Select one initial solution S_0 randomly in $\mathcal{PS}$. Initialise $bestsofar = S_0$ and $bestobj.C_{\max} = bestobj.TEC = Inf$. Let $iter = 1$ be the iteration count.
5: **while** $iter \leq I_{\max}$ **do**
6: Generate N_{cd} neighbour solutions on the basis of S_0 by following the **GetNeighbor** procedure (Algorithm 4.6). Let $\mathcal{CS}$ be the set of these solutions.
7: Let $\mathcal{P}$ be the set of $N_{cd}/2$ randomly selected Pareto solutions from $\mathcal{CS}$.
8: Let $Flag = 0$ be the binary indicator of finding a solution which dominates the *bestsofar* or not.
9: **for** $s = 1 : N_{cd}/2$ **do**
10: **if** *bestsofar* is dominated by $\mathcal{P}_s$ **then**
11: $bestobj.C_{\max} = \mathcal{P}_s.C_{\max}$, $bestobj.TEC = \mathcal{P}_s.TEC$.
12: $S_0 = \mathcal{P}_s$.
13: $bestsofar = S_0$.
14: Decrease the tabu length of every element in $\mathcal{TB}$ by one, if $|\mathcal{TB}| \neq 0$.
15: **if** $(\mathcal{P}_s.C_{\max}, \mathcal{P}_s.TEC) \in \mathcal{TB}$ **then**
16: Set the tabu length of $\mathcal{P}_s$ to be the maximum tabu length, i.e., L_{tb}.
17: **else**
18: $\mathcal{TB} = \mathcal{TB} \cup \{(\mathcal{P}_s.C_{\max}, \mathcal{P}_s.TEC)\}$.
19: Reset the tabu length of $\mathcal{P}_s$ by following Line 16.
20: **end if**
21: $Flag = 1$.
22: Break.
23: **end if**
24: **end for**
25: **if** $Flag = 0$ **then**
26: **for** $s = 1 : N_{cd}/2$ **do**
27: **if** $(\mathcal{P}_s.C_{\max}, \mathcal{P}_s.TEC) \in \mathcal{TB}$ and the tabu length of $\mathcal{P}_s$ is equal to zero **then**
28: $S_0 = \mathcal{P}_s$.
29: Decrease the tabu length of elements in $\mathcal{TB}$ by following Line 14.
30: Reset the tabu length of $\mathcal{P}_s$ by following Line 16.
31: **else if** $(\mathcal{P}_s.C_{\max}, \mathcal{P}_s.TEC) \notin \mathcal{TB}$ **then**
32: $S_0 = \mathcal{P}_s$.
33: Decrease the tabu length of elements in $\mathcal{TB}$ by following Line 14.
34: Update the objectives of $\mathcal{P}_s$ into $\mathcal{TB}$ and reset tabu length by following Lines 18- -19.
35: **end if**
36: **end for**
37: **end if**
38: $iter = iter + 1$.
39: $\mathcal{NS} = \mathcal{NS} \cup \mathcal{P}$.
40: **end while**
41: Delete the dominated solutions in $\mathcal{NS}$, if any.
42: Output: the set of Pareto solutions, $\mathcal{NS}$.

For job j, the pheromone in the arc from the last node to the node (j, i, ST_j^1) is denoted by $\tau_j(i,t)$ $(\forall j \in \mathcal{J}, i \in \mathcal{M}, t \in \mathcal{T})$, which is initialised to be a non-negative constant. Equations (4.1) – (4.2) define the updating of $\tau_j(i,t)$ in each iteration.

$$\Delta\tau_j^k(i,t) = \begin{cases} \sigma/TEC_k, & \text{ant } k \text{ travels the node } (j, i, ST_j^1) \quad (4.1) \\ 0, & \text{otherwise} \quad (4.2) \end{cases}$$

$$\Delta\tau_j(i,t) = \sum_{k=1}^{K} \Delta\tau_j^k(i,t) \quad (4.3)$$

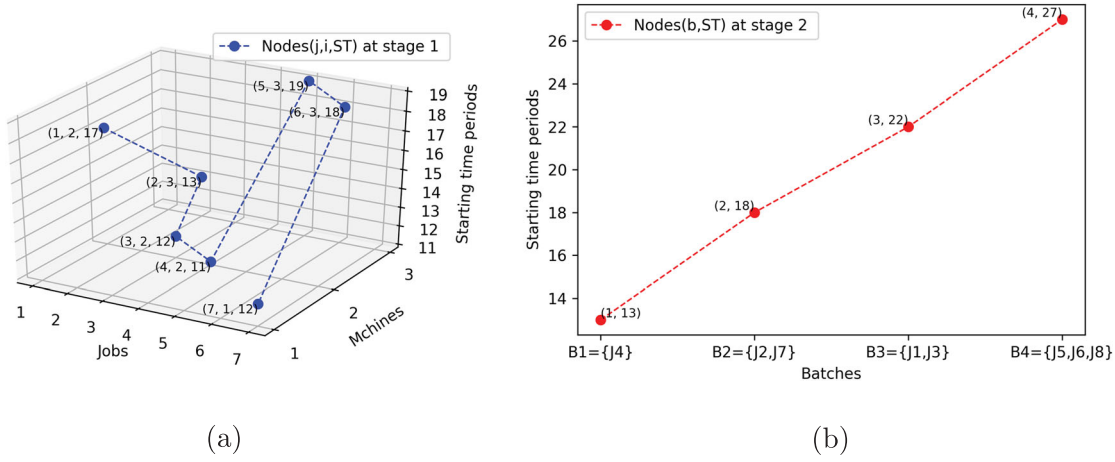

Figure 8. Nodes of stage 1 (a) and stage 2 (b) in bi-objective ACO for the solution presented in Figure 4.

$$\tau_j'(i,t) = (1-\rho)\tau_j(i,t) + \Delta\tau_j(i,t) \tag{4.4}$$

where K is the number of ants, $\Delta\tau_j^k(i,t)$ represents the increment of pheromone when the kth ant travels from the last node to the node (j,i,ST_j^1) in current iteration. σ is a constant that reflects on the strength of pheromone. TEC_k denotes the total energy consumption of kth ant's route, and ρ describes the evaporation speed of pheromone. $\tau_j'(i,t)$ is the amount of pheromone from the last node to the node (j,i,ST_j^1) in the next iteration.

4.3.3. The construction of ant solutions

The main idea is to search for the next node (j,i,ST_j^1) with the guidance of pheromone. In more details, for job j, there are many available nodes that the ant could arrive from the last node. The more pheromone retained in the arc from the last node to current node (j,i,ST_j^1), the higher possibility that ant will choose this arc during the next movement. The pseudo code of constructing the ant solution is shown in Algorithm 4.8.

Algorithm 4.8 ConstructAntSolution procedure

1: Input: the information of jobs, machines, energy prices, the weight of pheromone α, the weight of heuristic function β, the weight of pheromone volatilisation ρ, the strength of pheromone σ.
2: Let $\mathcal{J}_k(j)$ be the set of available nodes that ant k could travel for processing a certain job j.
3: Define the heuristic function of node (j,i,ST_j^1):
$\eta_j(i,t) = \sigma/TECP_{i,j}^t$, if the total number of jobs $n < 40$, where $TECP_{i,j}^t$ is the total energy consumption of partial route for scheduling jobs $1-j$.
$\eta_j(i,t) = \sigma/C\max_{i,j}^t$, if the total number of jobs $n \geq 40$, where $C\max_{i,j}^t$ is the makespan of partial route for scheduling jobs $1-j$.
4: Define the state transition probability between nodes for ant_k: $p_j^k(i,t) = \frac{[\tau_j(i,t)]^\alpha[\eta_j(i,t)]^\beta}{\sum_{s\in\mathcal{J}_k(j)}[\tau_j(s)]^\alpha[\eta_j(s)]^\beta}$, where the pheromone in the arc form the last node to the node (j,i,ST_j^1), $\tau_j(i,t)$, is given in Equation (4.4).
5: Sort all of n jobs randomly.
6: Select a node randomly for the first job (i.e., $j=1$) from the set of available nodes $\mathcal{J}_k(1)$.
7: **for** every job, j ($j \in \mathcal{J}\backslash\{1\}$) **do**
8: Compute the transition probability between nodes according to the Equation in Line 4.
9: Select the node for job j (i.e., determine the machine assignment and starting time of j at stage 1) using the roulette wheel strategy with the obtained transition probability to construct a solution, incrementally.
10: **end for**
11: The ant solution at stage 1 is constructed.
12: Form and schedule batches at stage 2 according to **BatchFormingRule** with $l=2$ in Algorithm 4.3, and **BatchSchedulingRule** in Algorithm 4.4.
13: A new ant solution s is constructed completely.
14: Output: s.

4.3.4. The complete bi-objective ant colony optimisation algorithm

The complete bi-objective ant colony optimisation algorithm is given in Algorithm 4.9. Lines 1–2 give the parameters and initialise the algorithm. Lines 3–8 initialise the pheromone in arcs. Lines 9–16 are the main loop of the bi-objective ACO algorithm. During every iteration, each ant is generated in Line 11, and updated with the guide of pheromone in Lines 12. The non-dominated solutions obtained in current iteration are preserved into the set $\mathcal{R}_{iter}$ in Line 14. After $G_{\max}$ iterations, all sets of non-dominated solutions are combined, and the dominated solutions are deleted in Lines 17–18. Finally, Line 19 outputs the set of Pareto solutions.

Algorithm 4.9 The bi-objective ant colony optimisation procedure

1: Input: the information of jobs, machines, energy prices, the number of ants K, the number of iterations $G_{\max}$, and other parameters of ACO including α, β, ρ and σ.
2: Let $\mathcal{NS} = \emptyset$ be the set of non-dominated solutions. Let matrix *Tau* records the pheromone in the arc between each pair of nodes.
3: **if** the number of jobs $n < 40$: **then**
4: Initialise the pheromone in the arc between each pair of nodes to be $\sigma/1000$.
5: **else**
6: Generate a set of Pareto solutions $\mathcal{PS}$ with the **constructive heuristic method** in Algorithm 4.1.
7: Update the matrix of pheromone *Tau* using the information of solutions in set $\mathcal{PS}$, according to Equations (4.1)−(4.4).
8: **end if**
9: **while** $iter \leq G_{\max}$ **do**
10: **for** each ant, k ($k \in \mathcal{K}$) **do**
11: Construct an ant solution ant_k with the **ConstructAntSolution** procedure in Algorithm 4.8.
12: Update the pheromone between nodes (i.e., *Tau*) with information in ant_k according to Equations (4.1)−(4.4).
13: **end for**
14: Select the non-dominated solutions obtained in the current iteration and update them into set $\mathcal{R}_{iter}$.
15: $iter = iter + 1$.
16: **end while**
17: Let $\mathcal{NS} = \bigcup_{iter=1}^{G_{\max}} \mathcal{R}_{iter}$.
18: Delete the dominated solutions in set $\mathcal{NS}$, if any.
19: Output: the set of Pareto solutions, $\mathcal{NS}$.

5. Computational experiments

In this section, some performance metrics for bi-objective optimisation are introduced. The Chess Rating System (CRS) (Veček et al. 2016) is used for parameter tuning for algorithms. Extensive computational experiments are conducted to evaluate the performance of the proposed algorithms. A case study is given. The AUGMECON method is coded with Python 3.6 and is run with Gurobi 7.5. The constructive heuristic method, bi-objective tabu search algorithm, and bi-objective ant colony optimisation algorithm are coded in MATLAB R2016(b). All methods are run on a PC with a 3.2 GHz Intel core i5-6500 processor and 8 GB RAM in Windows 7 system.

5.1. Performance metrics

For a multi-objective optimisation, it is difficult to evaluate the performance by a single metric, since the convergence to the Pareto-optimal set and the maintenance of diversity in solutions' distribution space are both vital criterions (Deb et al. 2002). In this paper, three performance metrics are as follows.

(1) The number of non-dominated solutions, *Nd*. In general, the more non-dominated solutions that a algorithm finds, the more effective this algorithm is.
(2) Distribution spacing metric, *DS* (Tan et al. 2006). This metric shows the diversity of the non-dominated solutions distributed along the Pareto front, which is defined in Equation (5.1).

$$DS = \frac{1}{\overline{D}}\sqrt{\frac{1}{|\mathcal{NS}|}\sum_{s\in\mathcal{NS}}(D_s - \overline{D})^2} \tag{5.1}$$

where $\mathcal{NS}$ is the set of non-dominated solutions. D_s denotes the Euclidean distance between solution $s \in \mathcal{NS}$ and its nearest neighbour solution, and $\overline{D} = \sum_{s\in\mathcal{NS}} D_s/|\mathcal{NS}|$. A smaller value of *DS* indicates a well-proportioned distribution of solutions along the Pareto front.

(3) Convergence metric, D_R (Ishibuchi, Yoshida, and Murata 2003). This metric is used to evaluate the distance of a set of non-dominated solutions to a reference front (i.e. the Pareto-optimal front or a near Pareto-optimal front). Let $\mathcal{S}^*$ be the reference solution set. If the optimal front is not known, all the non-dominated solutions are selected to form the set $\mathcal{S}^*$. D_R of a front $\mathcal{NS}$ is defined in Equation (5.2).

$$D_R(A) = \frac{1}{|\mathcal{S}^*|} \sum_{y \in \mathcal{S}^*} \min\{d_{xy} : x \in \mathcal{NS}\} \tag{5.2}$$

where d_{xy} is the Euclidean distance between a solution $x \in \mathcal{NS}$ and a solution $y \in \mathcal{S}^*$. In this problem, d_{xy} can be calculated by Equation (5.3).

$$d_{xy} = \sqrt{(f_1^*(x) - f_1^*(y))^2 + (f_2^*(x) - f_2^*(y))^2} \tag{5.3}$$

where $f_1^*(\cdot)$ and $f_2^*(\cdot)$ represents the objective values of the $C_{\max}$ and *TEC*, respectively, which are normalised by using the solutions in the reference front $\mathcal{S}^*$ with Equation (5.4).

$$f_i^*(\cdot) = \frac{f_i(\cdot) - f_{\min_i}(\mathcal{S}^*)}{f_{\max_i}(\mathcal{S}^*) - f_{\min_i}(\mathcal{S}^*)} \tag{5.4}$$

where $f_i(\cdot)$ represents the ith objective value of the solution $(\cdot)$, $i = \{1, 2\}$. $f_{\max_i}(\mathcal{S}^*)$ and $f_{\min_i}(\mathcal{S}^*)$ represents the maximum and minimum values of ith objective in the reference front $\mathcal{S}^*$, respectively. The smaller value of D_R is, the better convergence of set $\mathcal{NS}$ presents.

5.2. Parameter tuning

The value of parameters will affect the performance of meta-heuristic algorithms significantly. Therefore, appropriate methods should be adopted to tune the parameters of proposed bi-objective tabu search and bi-objective ant colony optimisation algorithm.

Chess rating system (CRS) is a novel tuning method proposed by Veček et al. (2016). It is designed to find good parameters configurations within a framework of evolutionary algorithm. The main idea of CRS tuning is to generate new parameter configurations by crossover and mutation procedure, and conduct experiments in form of a tournament. In more details, every parameter configuration is a chess player, and they will solve a set of optimisation problems over a number of independent runs for obtaining scores. With the obtained scores, a rating value (as well as its deviation and confidence interval with significance level of 5%) could be calculated, which reflects the performance of incumbent parameter configuration.

5.2.1. Generate testing problems for parameter tuning

A set of sample problems is needed for testing the performance of an algorithm under different parameter configurations. For the investigated problem, the following ten parameters that determine a detailed problem are considered: (1) number of jobs, n; (2) number of machines at stage 1, m; (3) capacity of the furnace at stage 2, Q; (4) proportion of machines with eligibility for processing each job, R; (5) the TOU energy price, c; (6) time periods, T, with lower bound $T \geq \max\{\sum_{j=1}^{n} p_j^1/0.8/m, \max\{p_j^1\}\} + \sum_{j=1}^{n} q_j/Q \times \min\{p_j^2, j = 1, 2, \ldots, n\}$; (7) the range of processing time at two stages p_j^1, p_j^2: $U[p_{\min}^1, p_{\max}^1]$ and $U[p_{\min}^2, p_{\max}^2]$; (8) the range of size of jobs q_j: $U[q_{\min}, q_{\max}]$; (9) the range of energy consumption rate for parallel machines in working and idle states e_i^1, e_i^2: $U[e_{\min}^1, e_{\max}^1]$ and $U[e_{\min}^2, e_{\max}^2]$; (10) the range of energy consumption rate for the furnace in working and idle states E^1, E^2: $U[E_{\min}^1, E_{\max}^1]$ and $U[E_{\min}^2, E_{\max}^2]$, where U represents the discrete uniform distribution. The parameters for sample problems are set in Tables 3 and 4. Two kinds of TOU energy price c^1, c^2 are given in Equation (5.5). The combination of above parameters results in a set of $5 \times 3 \times 2 \times 3 \times 2 = 180$ sample problems. For each problem, one instance is randomly generated and solved for parameter tuning.

$$c^1(t) = \begin{cases} U[10, 13], & 1 \leq t \leq \lceil T/4 \rceil \\ U[5, 7], & \lceil T/4 \rceil + 1 \leq t \leq \lceil T/2 \rceil \\ U[4, 6], & \lceil T/2 \rceil + 1 \leq t \leq \lceil 3T/4 \rceil \\ U[3, 5], & \lceil 3T/4 \rceil + 1 \leq t \leq T \end{cases}$$

Table 3. Problems configuration for parameter tuning (Part I).

Parameters	Set of values
n	{20, 30, 40, 50, 100}
m	{10, 15, 20}
Q	{7,10}
R	{0.6, 0.8, 1}
c	$\{c^1, c^2\}$

Table 4. Problems configuration for parameter tuning (Part II).

Parameters	Value
T	$\max\{\lceil 4n/m\rceil, 30, \min\{5n, 300\}\}$
$q_j \sim U[q_{\min}, q_{\max}]$	[2, 4]
$p_j^1 \sim U[p_{\min}^1, p_{\max}^1]$	[1, 3]
$p_j^2 \sim U[p_{\min}^2, p_{\max}^2]$	[2, 4]
$e_i^1 \sim U[e_{\min}^1, e_{\max}^1]$	[3, 5]
$e_i^2 \sim U[e_{\min}^2, e_{\max}^2]$	[1, 2]
$E^1 \sim U[E_{\min}^1, E_{\max}^1]$	[4, 6]
$E^2 \sim U[E_{\min}^2, E_{\max}^2]$	[2, 3]

$$c^2(t) = \begin{cases} U[12,14], & 1 \le t \le \lceil T/4\rceil \\ U[7,9], & \lceil T/4\rceil + 1 \le t \le \lceil T/2\rceil \\ U[5,8], & \lceil T/2\rceil + 1 \le t \le \lceil 3T/4\rceil \\ U[3,5], & \lceil 3T/4\rceil + 1 \le t \le T \end{cases} \tag{5.5}$$

5.2.2. *Candidate levels of parameter in algorithms*

For the bi-objective tabu search algorithm, the following two parameters should be considered:

(1) Length of tabu list: *TabuL*;
(2) Number of candidate solutions at each iteration: *CaNum*.

For the bi-objective ant colony optimisation algorithm, four parameters should be tuned:

(1) Number of ants: K;
(2) Weight of pheromone: α;
(3) Weight of heuristic function: β;
(4) Weight of pheromone volatilisation: ρ.

The candidate levels of above parameters are determined through preliminary experiments. The detailed candidate values of parameter for bi-objective tabu search and ant colony optimisation algorithms are shown in Table 5, where 'm' and 'n' represent the number of machines at stage 1 and the number of jobs, respectively. For fair comparisons, the maximum number of iterations of tabu search algorithm and ant colony optimisation algorithm, $I_{\max}$ and $G_{\max}$, are set to be equal: $I_{\max} = G_{\max} = 100$.

5.2.3. *Parameters tuning with CRS*

According to the CRS tuning method, there are six parameters that should be determined before the tuning procedure begins: (1) Maximum number of execution times $ET_{\max}$, (2) Population size M, (3) Maximum parent size MPS, (4) Probability of crossover P_c, (5) Probability of mutation P_m, and (6) Number of repetition times for solving one problem N_r. In this paper, they are set as follows: $ET_{\max} = 1000$, $M = 50$, $MPS = M/2 = 25$, $P_c = 0.6$, $P_m = 0.4$, and $N_r = 3$. The detailed procedure of CRS tuning for the algorithms is described in Veček et al. (2016).

Table 5. The candidate values of parameters for two algorithms.

Algorithm	Parameters	Candidate values
Bi-objective tabu search	*TabuL*	$\{mn/8, mn/5, mn/4, mn/3, mn/2\}$
	CaNum	$\{2n/m, 3n/m, 4n/m\}$
Bi-objective ant colony optimisation	K	$\{10, 30, 50, 80\}$
	α	$\{1, 3, 5, 8, 10\}$
	β	$\{1, 4, 6, 8, 10\}$
	ρ	$\{0.05, 0.1, 0.3, 0.5\}$

Table 6. The results of CRS tuning for the bi-objective tabu search algorithm.

Top five configurations (*TabuL*, *CaNum*)	Rating	Rating deviation	Rating interval
(mn/4, 2n/m)	**1560.91**	129.02	[1431.89, 1689.93]
$(mn/8, 2n/m)$	1554.49	123.23	[1431.26, 1677.72]
$(mn/3, 2n/m)$	1467.94	126.31	[1341.63, 1594.25]
$(mn/3, 4n/m)$	1442.30	131.04	[1311.26, 1573.34]
$(mn/4, 2n/m)$	1294.84	123.90	[1170.94, 1418.74]

Table 7. The results of CRS tuning for the bi-objective ant colony optimisation algorithm.

Top five configurations (K, α, β, ρ)	Rating	Rating deviation	Rating interval
(50, 5, 4, 0.1)	**1532.21**	134.32	[1397.89, 1666.53]
(50, 8, 8, 0.1)	1525.45	136.47	[1388.98, 1661.92]
(50, 5, 4, 0.1)	1501.36	142.38	[1358.98, 1643.74]
(50, 5, 4, 0.1)	1483.02	144.56	[1338.46, 1627.58]
(50, 5, 4, 0.1)	1389.31	132.35	[1256.96, 1521.66]

Note that the proposed CRS tuning system in Veček et al. (2016) focuses mainly on algorithms to optimise single objective. Hence, for the investigated bi-objective problem, performance metric D_R (see Section 5.1) is adopted, and configuration with smaller D_R will win the tournament.

The results of the parameters tuning are given in Tables 6 and 7. According to the CRS tuning system, for a certain algorithm, the largest rating of the obtained configurations is the best one. Hence, we can find the most appropriate parameters configuration and their rating for the bi-objective tabu search algorithm and bi-objective ant colony optimisation algorithm, respectively, as suggested in bold in Tables 6 and 7. To reduce redundancy, only the top five parameter configurations are listed in these tables.

5.3. Computational results

5.3.1. Data generation

As is mentioned in the parameters tuning, the parameters related to the investigated problem include n, m, Q, c, R and all of the parameters listed in Table 4 (see Section 5.2.1). The size of problems is denoted as $[n, m, Q]$, with three representative parameters (i.e. n,m and Q).

The combinations of above parameters are considered. For small-scale problems: $[n, m, Q]$: $n \in \{5, 8, 10, 15, 20\}$, $m \in \{5, 6, 7\}$, $Q \in \{5, 7\}$, with $c = c^1$, and for medium- and large-scale problems: $[n, m, Q, c]$: $n \in \{40, 50, 80, 100, 120\}$, $m \in \{30, 40, 50\}$, $Q \in \{10, 15\}$, with $c = c^2$, where c^1, c^2 are given in Equation (5.5). Hence, it results in $5 \times 3 \times 2 \times 1 = 30$ small-scale problems and 30 medium- and large-scale problems. Moreover, R is set to be 0.8, and all other parameters take the values suggested in Table 4 for both small-scale and medium- and large-scale problems. These 60 problems are numbered from 1 to 60. For every problem, five instances are randomly generated and tested. All the data of $60 \times 5 = 300$ problem instances and the related code can be downloaded via the following link: pan.baidu.com/s/1Xoqv4QDv_7l058NRVREuVA.

5.3.2. Comparison results for small-scale problem instances

The performance of augmented ϵ-constrain method (AUGMECON), constructive heuristic method (CH), bi-objective tabu search algorithm (TS) and bi-objective ant colony optimisation algorithm (ACO) are compared. In the AUGMECON

Table 8. Comparison results for small-scale problem instances.

Problem [n, m, Q]				$\overline{Nd}$				$\overline{DS}$				$\overline{D_R}$				$\overline{CPUs}$			
No.	n	m	Q	AUG.	CH	TS	ACO	AUG.	CH	TS	ACO	AUG.	CH	TS	ACO	AUG. (gap %)	CH	TS	ACO
1	5	5	5	18.0	8.4	14.0	10.0	0.59	0.81	0.63	1.01	0.00	0.03	0.05	0.06	24.3(0)	5.2	9.1	35.5
2			7	20.0	7.6	14.4	9.0	0.77	1.10	0.99	1.48	0.01	0.04	0.04	0.06	66.5(0)	6.6	11.5	33.7
3		6	5	19.4	6.4	10.6	8.0	0.94	1.22	0.92	1.01	0.01	0.05	0.11	0.07	42.0(0)	9.4	15.3	44.4
4			7	20.0	8.0	13.0	9.6	0.81	0.72	0.82	1.12	0.01	0.04	0.06	0.06	52.5(0)	12.1	18.6	43.0
5		7	5	18.4	7.6	13.0	11.4	0.70	1.26	0.84	1.00	0.00	0.06	0.09	0.10	54.3(0)	14.4	20.4	53.8
6			7	19.6	6.6	14.0	9.6	0.82	1.07	0.77	1.25	0.01	0.04	0.05	0.06	63.6(0)	15.8	23.0	53.1
	Average			**19.2**	7.4	13.2	9.6	**0.77**	1.03	0.83	1.15	**0.01**	0.04	0.07	0.07	**50.5(0)**	**10.6**	**16.3**	**43.9**
7	8	5	5	19.0	3.0	8.0	9.4	0.53	0.85	1.00	1.38	0.01	0.02	0.06	0.06	617.1(0)	6.4	11.3	90.2
8			7	26.4	12.4	17.0	7.0	0.78	1.06	0.91	0.85	0.00	0.05	0.06	0.06	378.8(0)	5.3	11.0	84.1
9		6	5	21.6	2.4	7.0	9.4	0.58	1.06	1.33	0.92	0.01	0.02	0.06	0.09	851.6(0)	7.1	12.4	114.6
10			7	25.4	11.6	15.0	8.0	0.74	1.30	0.97	1.52	0.01	0.05	0.06	0.06	780.9(0)	7.4	14.1	109.9
11		7	5	18.6	6.4	12.0	10.4	0.63	0.97	0.78	1.07	0.01	0.03	0.08	0.12	663.7(0)	8.5	13.6	143.9
12			7	25.0	9.4	15.6	8.6	0.76	0.92	1.07	1.19	0.00	0.04	0.05	0.05	951.3(0)	11.1	15.9	141.0
	Average			**22.7**	7.5	12.4	8.8	**0.67**	1.03	1.01	1.15	**0.01**	0.03	0.06	0.07	**707.2(0)**	**7.6**	**13.0**	**114.0**
13	10	5	5	24.0	9.0	18.0	9.6	0.63	1.56	0.88	1.23	0.01	0.02	0.03	0.04	963.2(0)	20.0	32.0	146.0
14			7	31.4	12.6	16.0	6.4	0.77	1.31	0.94	0.98	0.01	0.03	0.04	0.04	3395.3(0)	21.3	33.3	133.6
15		6	5	24.0	3.6	16.0	11.4	0.57	1.07	0.85	1.41	0.01	0.02	0.07	0.05	730.6(0)	25.5	37.0	198.1
16			7	30.4	12.4	17.4	10.6	0.75	0.93	0.78	1.00	0.01	0.04	0.05	0.05	6945.2(0)	26.3	38.1	195.8
17		7	5	24.6	5.6	15.0	10.4	0.57	0.98	0.49	0.82	0.01	0.03	0.06	0.06	1277.5(0)	29.7	42.8	241.1
18			7	34.8	10.4	17.6	8.0	0.81	1.30	1.05	1.05	0.00	0.04	0.07	0.07	3031.9(0)	30.1	43.5	243.4
	Average			**28.2**	8.9	16.7	9.4	**0.68**	1.19	0.83	1.08	**0.01**	0.03	0.05	0.05	**2724.0(0)**	**25.5**	**37.8**	**193.0**
19	15	5	5	35.4	11.0	17.0	11.0	0.71	1.40	0.96	1.08	0.03	0.02	0.03	0.04	11280.8(0)	38.4	56.6	366.0
20			7	12.0	18.6	20.4	10.0	0.74	1.19	1.10	1.82	0.37	0.06	0.06	0.11	7605.8(6.7)	37.9	56.7	363.5
21		6	5	18.4	7.0	16.0	9.2	0.94	1.42	1.00	0.99	0.26	0.09	0.07	0.11	7839.6(1.4)	47.9	67.3	494.4
22			7	11.0	13.6	20.0	11.8	0.95	1.75	1.12	1.54	0.41	0.09	0.07	0.13	7419.2(9.9)	45.9	65.2	508.1
23		7	5	7.4	6.0	15.0	10.0	1.09	1.00	1.08	1.26	0.28	0.18	0.07	0.13	4978.8(15.4)	57.9	76.0	658.2
24			7	11.0	14.2	17.0	7.6	0.69	1.15	1.01	1.21	0.39	0.10	0.09	0.13	7542.0(6.4)	52.5	71.1	656.9
	Average			15.9	11.7	**17.6**	9.9	**0.85**	1.32	1.05	1.32	0.29	0.09	**0.06**	0.11	**7777.7(6.6)**	**46.7**	**65.5**	**507.8**
25	20	5	5	8.0	10.4	20.0	12.6	1.12	1.04	1.06	1.08	0.44	0.12	0.06	0.10	7456.4(2.0)	64.9	91.4	734.9
26			7	5.0	17.0	20.2	11.0	0.97	1.15	1.03	1.12	0.51	0.07	0.06	0.12	7510.6(16.0)	61.7	88.8	709.6
27		6	5	2.4	15.0	16.2	11.6	0.92	1.41	1.35	1.22	0.15	0.10	0.09	0.13	7349.2(40.6)	79.3	105.4	983.5
28			7	4.0	13.8	19.0	9.0	0.80	1.06	1.06	1.10	0.54	0.11	0.09	0.10	7201.1(33.1)	75.8	102.2	985.9
29		7	5	\	4.4	16.0	13.0	\	1.13	0.92	1.06	\	0.28	0.07	0.09	\	97.8	122.3	1196.2
30			7	\	10.6	19.4	8.8	\	0.83	1.15	0.76	\	0.08	0.06	0.08	\	88.3	113.1	1176.9
	Average			4.9	11.9	**18.5**	11.0	**0.96**	1.10	1.10	1.06	0.41	0.12	**0.07**	0.10	**7379.3(22.9)**	**78.0**	**103.9**	**964.5**

method, the time limit for finding each ideal point and solving every ϵ-constraint problems are set to be 3600 and 900 CPU seconds, respectively. The optimality gap value is 1×10^{-6}. If the time limit is reached before the optimality is proved, the incumbent solutions and their gap values will be recorded. The results are shown in Table 8. In the table, 'AUG'. stands for AUGMECON, '$\overline{CPUs}$' represents the computation time in CPU seconds. For each problem [n, m, Q], the criteria values of five instances are averaged. '\' means that the AUGMECON method cannot obtain any solution for this problem, since the ideal points are not found within the time limit. For each number of jobs n, the average value of the performance metrics for six problems is computed. If the AUGMECON cannot obtain solutions for some problems within the time limit, only the solved problems are considered to compute the average values.

The results show that when the number of jobs $n \leq 10$, AUGMECON can obtain the optimal Pareto front for the tested instances, with more non-dominated solutions than CH, TS and ACO. When the number of jobs is relatively large (i.e. $n \geq 15$), TS performs best in terms of $\overline{Nd}$. The AUGMECON could only obtain a part of solutions in (approximate) Pareto front, or even cannot obtain any solutions for problems with $n = 20, m = 7$. The gap of solution tends to increase with the increase of the problem scale. The $\overline{Nd}$ values of CH and ACO are relatively close for most instances.

In terms of '$\overline{CPUs}$', the computation time increases as the problem scale increases for all algorithms, as expected. Especially for the AUGMECON, it is rather time-consuming for solving problems with more than 15 jobs. While CH, TS and ACO could obtain approximate Pareto fronts much more faster. In more details, TS takes longer time than CH since the constructive heuristic is embedded into TS for initialisation (see Algorithm 4.7 Line 3). ACO is relative slower, this may be due to the reason that the calculation of state transition probability between nodes and updating of pheromone (see Algorithm 4.9 Lines 11-12) consumes a lot of computational efforts.

Moreover, for CH, TS and ACO, the close values of '$\overline{DS}$' indicates that Pareto fronts obtained by three algorithms have similar proportion of distribution. The AUGMECON performs always best in this criterion. As for the most important metric, '$\overline{D_R}$', which reflects the convergence of Pareto fronts, TS performs best when the exact Pareto front cannot be obtained.

Take an example, the Pareto fronts of problem #1 instance 3 and problem #12 instance 2 generated with four algorithms are given to illustrate their performance, as shown in Figures 9 and 10, respectively.

5.3.3. Comparison results for medium- and large-scale problem instances

Since the CH method has been integrated into two meta-heuristic algorithms (see Algorithm 4.7, Line 3, and Algorithm 4.9, Line 6), only the bi-objective TS and bi-objective ACO algorithms are adopted to tackle the medium- and large-scale

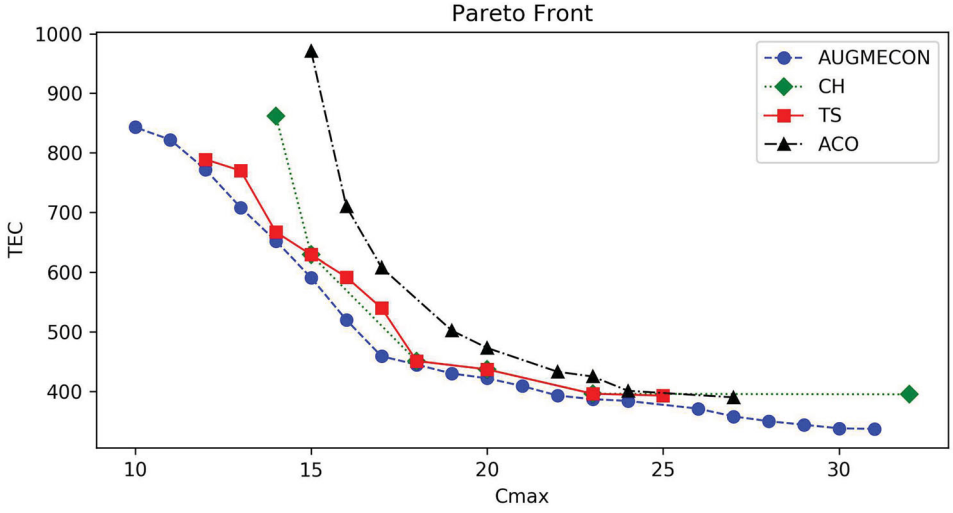

Figure 9. The sample Pareto fronts of problem #1 instance 3.

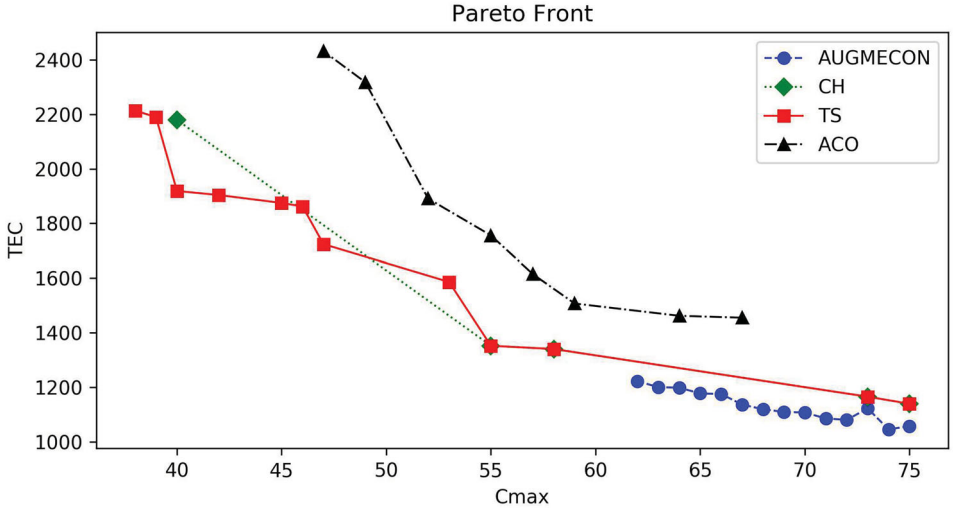

Figure 10. The sample Pareto fronts of problem #12 instance 2.

Table 9. Comparison results for medium- and large-scale problem instances.

Problem [n, m, Q]				$\overline{Nd}$		$\overline{DS}$		$\overline{D_R}$		$\overline{CPUs}$	
No.	n	m	Q	TS	ACO	TS	ACO	TS	ACO	TS	ACO
31	40	30	10	17.6	8.4	0.91	1.37	0.00	0.08	793.3	1480.8
32			15	13.2	10.8	1.56	1.37	0.00	0.02	719.2	1421.8
33		40	10	16.8	8.0	1.51	1.41	0.00	0.05	1212.7	2298.4
34			15	14.0	13.0	1.27	1.48	0.01	0.02	929.0	1992.4
35		50	10	13.8	10.0	1.33	1.59	0.01	0.04	1736.1	3072.1
36			15	15.6	13.0	0.91	1.06	0.01	0.02	1538.2	2904.5
	Average			**15.2**	10.5	**1.25**	1.38	**0.01**	0.04	**1154.8**	**2195.0**
37	50	30	10	18.0	8.0	1.08	1.41	0.01	0.10	1390.2	2045.6
38			15	14.6	14.0	1.30	1.15	0.02	0.02	1465.8	2445.6
39		40	10	16.0	10.6	1.04	1.94	0.01	0.08	2144.6	3596.9
40			15	13.2	13.0	1.51	1.40	0.01	0.02	2121.3	3670.8
41		50	10	17.6	10.0	0.97	1.58	0.02	0.09	3353.3	6016.3
42			15	14.2	12.0	1.47	1.47	0.01	0.02	3202.4	5549.4
	Average			**15.6**	11.3	**1.23**	1.49	**0.01**	0.05	**2279.6**	**3887.4**
43	80	30	10	21.2	11.6	1.36	1.81	0.00	0.08	3433.0	3773.0
44			15	14.2	10.2	1.26	1.74	0.00	0.07	3503.6	3765.6
45		40	10	15.2	5.6	1.67	1.26	0.00	0.10	6482.7	6135.7
46			15	12.2	8.6	1.07	1.28	0.00	0.05	5351.5	5357.0
47		50	10	11.2	8.8	1.27	1.32	0.01	0.03	9179.0	9484.1
48			15	10.6	10.2	0.90	1.75	0.02	0.07	7568.6	8132.5
	Average			**14.1**	9.2	**1.25**	1.53	**0.00**	0.07	**5919.7**	**6108.0**
49	100	30	10	19.2	8.2	1.16	1.99	0.01	0.21	4417.1	6160.3
50			15	15.6	10.0	1.28	1.83	0.01	0.07	4199.6	5552.8
51		40	10	25.6	7.0	1.25	1.65	0.02	0.22	6973.1	9779.6
52			15	19.4	13.4	1.36	2.09	0.01	0.08	6111.3	8466.0
53		50	10	14.2	9.2	1.33	1.95	0.01	0.09	11,365.7	16,023.8
54			15	14.6	12.0	1.39	1.99	0.01	0.05	9205.2	12,561.5
	Average			**18.1**	10.0	**1.29**	1.92	**0.01**	0.12	**7045.3**	**9757.3**
55	120	30	10	18.0	11.2	1.27	1.04	0.00	0.04	8458.9	12,237.8
56			15	26.6	21.4	0.97	0.97	0.00	0.01	4334.5	8498.2
57		40	10	13.0	10.0	1.22	1.11	0.00	0.01	11,536.8	14,389.5
58			15	20.6	15.6	1.25	1.14	0.00	0.02	6814.0	10,715.0
59		50	10	12.6	9.6	1.16	1.05	0.00	0.01	16,877.3	20,439.1
60			15	17.4	13.4	1.15	1.00	0.00	0.01	10,669.4	14,896.5
	Average			**18.0**	13.5	1.17	**1.05**	**0.00**	0.02	**9781.8**	**13,529.4**

problems. For each problem, five instances are solved and the average value of the performance metrics are recorded. The fronts obtained by these two algorithms will be combined, and all the non-dominated solutions are selected to form the set S^* for computing the D_R metric. The results are shown in Table 9.

The results show that TS performs overall better than ACO. More specifically, TS could find more non-dominated solutions than ACO with better convergency in less computation time, since TS performs better in terms of '$\overline{Nd}$', '$\overline{D_R}$' and '$\overline{CPUs}$'. For most problem instances, the '$\overline{DS}$' metric of TS is smaller, which indicates that it conducts more well-proportioned Pareto fronts compared to ACO. Due to the complexity of the investigated problem, the required computation time to solve large-scale problems (i.e.$n \geq 100$) is relatively large for both two algorithms.

As an example, Figures 11 and 12 illustrate the approximate Pareto fronts achieved by these two algorithms for problem #39 instance 3 and problem #52 instance 1, respectively.

5.3.4. A case study of glass production

A real-world case originated from a glass-ceramics production system is given to validate the effectiveness of the proposed algorithms. In this case, there are five parallel machines at Stage 1 and a furnace with a capacity of 24 at Stage 2. The detailed

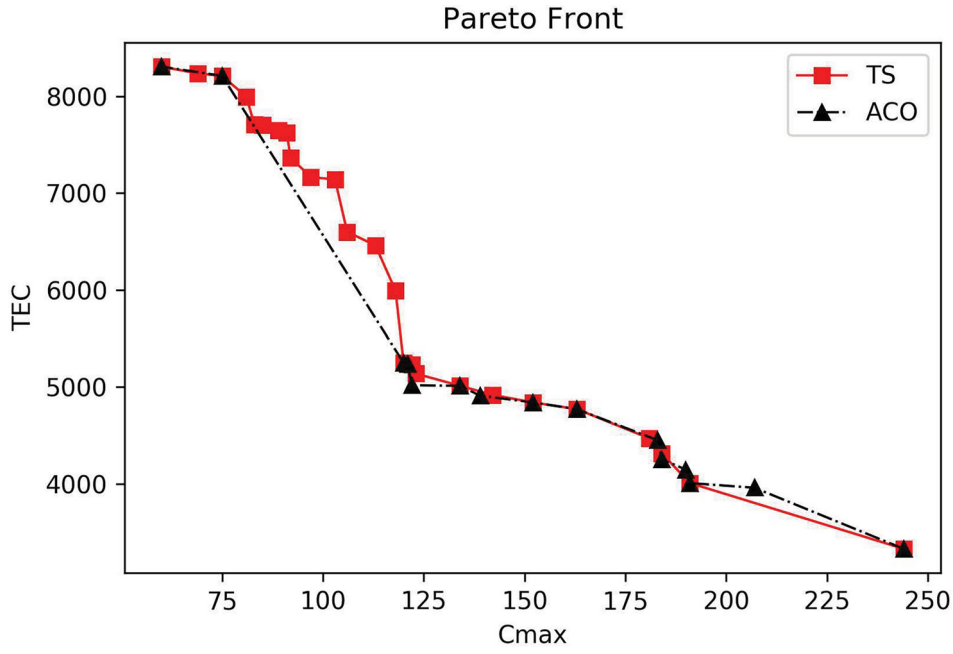

Figure 11. The approximate Pareto fronts of problem #39 instance 3.

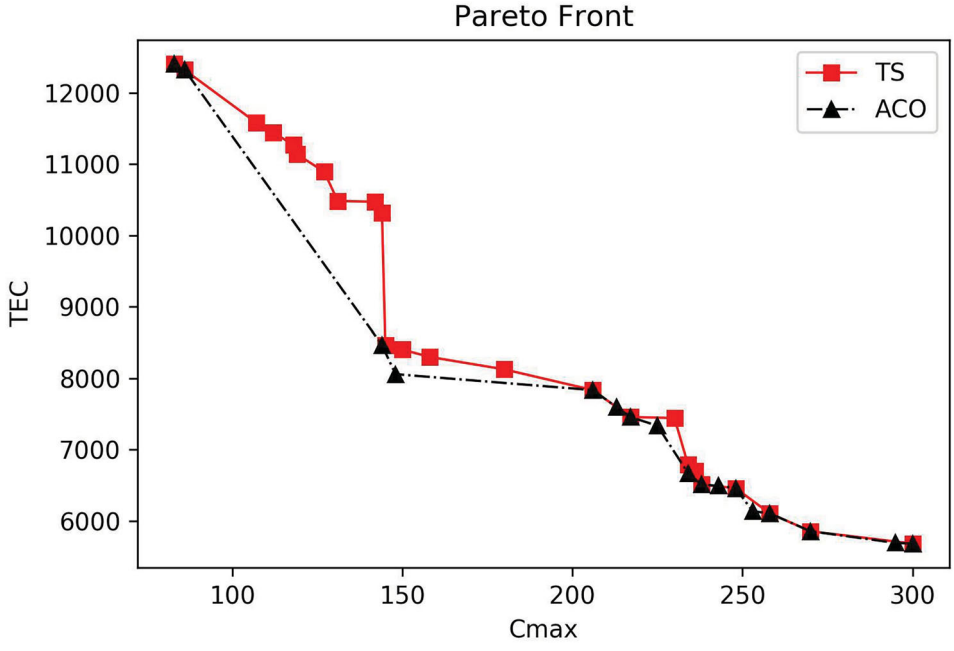

Figure 12. The approximate Pareto fronts of problem #52 instance 1.

machine-related and processing-related information is given in Tables A.1 and A.2, respectively. As shown in Table A.2, there are 50 jobs. The processing times on machines and the furnace are given in the table. The size of jobs is also given. If a job is not allowed to be processed on a machine, it is labelled by '\' in the corresponding cell. After preprocessing these data for commercial confidentiality issues, common characteristics of the glass-ceramics production have been preserved. The TOU energy prices during the non-summer time in Shanghai, China are adopted, which is shown in Figure A.1. The volume of usage is equal to 35 kilovolt in this case. The proposed bi-objective tabu search and ant colony optimisation algorithms are applied with the same parameters as in Section 5.3. The obtained Pareto solutions are listed in Table 10, where '\' means that there is no more solutions.

Table 10. The results of the case obtained by two algorithms (unit of C_{max}: 10 min, unit of *TEC*: RMB Yuan).

Bi-objective TS			*Bi-objective ACO*		
Solution ID	C_{max}	*TEC*	*Solution ID*	C_{max}	*TEC*
1	90	12,998.59	1	90	12,998.59
2	110	12,963.95	2	113	12,585.32
3	113	12,585.32	3	122	12,338.43
4	119	12,564.87	4	209	11,527.57
5	122	12,338.43	5	239	11,408.06
6	209	11,527.57	\	\	\
7	239	11,408.06	\	\	\

As shown in Table 10, it takes bi-objective ACO algorithm 1654.6 seconds to generate 5 non-dominated solutions, whereas the bi-objective TS algorithm can obtain 7 non-dominated solutions in 1020.8 s. Since 5 non-dominated solutions obtained by the bi-objective ACO algorithm are exactly covered by the solutions obtained by the bi-objective TS algorithm in less computation efforts, TS outperforms ACO for the case. Overall, solutions of this case range from total energy costs 11408.06 (RMB Yuan) to 12998.59 (RMB Yuan) and makespan from 90 to 239, which indicates a marginal energy cost saving of 10.67 RMB Yuan with the makespan increment of one unit time. It can be used for production managers to tradeoff the total energy cost and the makespan.

6. Conclusions and future work

Inspired by a real-world glass manufacturing system, this paper investigates a bi-objective energy-efficient two-stage hybrid flow shop scheduling problem with multiple parallel machines at stage 1, and a single batch machine at stage 2. The two objectives are to minimise the makespan and to minimise the total energy consumption simultaneously. The machine eligibility, different energy consumption rates in different working states (i.e. working, idle and turnoff) of machines, batch forming, and the time-of-use energy price are considered together. Based on the proposed MIP mathematical formulation, the augmented ϵ-constraint method is adopted to obtain exact Pareto fronts for small-scale problems. A problem-tailored constructive heuristic method with local search strategy is proposed. A bi-objective tabu search algorithm and a bi-objective ant colony optimisation algorithm are designed to tackle medium- and large-scale problems, in which the constructive heuristic method is integrated. The parameters of these algorithms are tuned by the Chess Rating System. Extensive computational experiments are conducted and a real-world case is solved. The results illustrate the promising performance of the proposed algorithms, in particular, the bi-objective tabu search.

For the future work, first, the property of this problem could be further explored. Algorithms integrated with possible problem properties could be developed to generate Pareto fronts more quickly. Second, more realistic factors could be integrated into this problem. For instance, the energy-efficient hybrid flow-shop scheduling with periodic maintenance or sequence-dependent setup times could be investigated as a valuable extension.

Disclosure statement

No potential conflict of interest was reported by the authors.

Funding

This work was supported by the National Science Foundation of China (NSFC) [grant no. 71571135]. The work was also supported by the Fundamental Research Funds for the Central Universities.

References

Abikarram, J. B., K. McConky, and R. Proano. 2019. "Energy Cost Minimization for Unrelated Parallel Machine Scheduling Under Real Time and Demand Charge Pricing." *Journal of Cleaner Production* 208: 232–242.

Aghelinejad, M. M., Y. Ouazene, and A. Yalaoui. 2018a. "Production Scheduling Optimisation with Machine State and Time-dependent Energy Costs." *International Journal of Production Research* 56 (16): 5558–5575.

Aghelinejad, M. M., Y. Ouazene, and A. Yalaoui. 2018b. "Energy Efficient Scheduling Problems Under Time-of-use Tariffs with Different Energy Consumption of the Jobs." *IFAC-PapersOnLine* 51 (11): 1053–1058.

Bérubé, J. F., M. Gendreau, and J. Y. Potvin. 2009. "An Exact ϵ-constraint Method for Bi-objective Combinatorial Optimization Problems: Application to the Traveling Salesman Problem with Profits." *European Journal of Operational Research* 194 (1): 39–50.

Bruzzone, A. A. G., D. Anghinolfi, M. Paolucci, and F. Tonelli. 2012. "Energy-aware Scheduling for Improving Manufacturing Process Sustainability: A Mathematical Model for Flexible Flow Shops." *CIRP Annals* 61: 459–462.

Che, A. D., X. Q. Wu, J. Peng, and P. Y. Yan. 2017. "Energy-efficient Bi-objective Single-machine Scheduling with Power-down Mechanism." *Computers & Operations Research* 85: 172–183.

Che, A. D., Y. Z. Zeng, and K. Lyu. 2016. "An Efficient Greedy Insertion Heuristic for Energy-conscious Single Machine Scheduling Problem Under Time-of-use Electricity Tariffs." *Journal of Cleaner Production* 129: 565–577.

Che, A. D., S. B. H. Zhang, and X. Q. Wu. 2017. "Energy-conscious Unrelated Parallel Machine Scheduling Under Time-of-use Electricity Tariffs." *Journal of Cleaner Production* 156: 688–697.

Chen, B., and X. D. Zhang. 2019. "Scheduling with Time-of-use Costs." *European Journal of Operational Research* 274 (3): 900–908.

Cheng, J. H., F. Chu, M. Liu, P. Wu, and W. L. Xia. 2017. "Bi-criteria Single-machine Batch Scheduling with Machine on/off Switching Under Time-of-use Tariffs." *Computers & Industrial Engineering* 112: 721–734.

Cui, W. W., and L. Li. 2018. "A Game-theoretic Approach to Optimize the Time-of-use Pricing Considering Customer Behaviors." *International Journal of Production Economics* 201: 75–88.

Dai, M., D. B. Tang, A. Giret, M. A. Salido, and W. D. Li. 2013. "Energy-efficient Scheduling for a Flexible Flow Shop Using An Improved Genetic-simulated Annealing Algorithm." *Robotics and Computer-Integrated Manufacturing* 29: 418–429.

Deb, K., A. Pratap, S. Agarwal, and T. Meyarivan. 2002. "A Fast and Elitist Multi-objective Genetic Algorithm: NSGA-II." *IEEE Transactions on Evolutionary Computation* 6 (2): 182–197.

Ding, J. Y., S. J. Song, and C. Wu. 2016. "Carbon-efficient Scheduling of Flow Shops by Multi-objective Optimization." *European Journal of Operational Research* 248 (3): 758–771.

Dorigo, M. 1992. "Optimization, Learning and Natural Algorithms." PhD thesis, Politecnico di Milano, Italie.

Fang, K. T., and B. M. T. Lin. 2013. "Parallel-machine Scheduling to Minimize Tardiness Penalty and Power Cost." *Computers & Industrial Engineering* 64 (1): 224–234.

Fang, K., N. A. Uhan, F. Zhao, and J. W. Sutherland. 2016. "Scheduling on a Single Machine Under Time-of-use Electricity Tariffs." *Annals of Operations Research* 238 (1): 199–227.

Gahm, C., F. Denz, M. Dirr, and A. Tuma. 2016. "Energy-efficient Scheduling in Manufacturing Companies: A Review and Research Framework." *European Journal of Operational Research* 248 (3): 744–757.

Giret, A., D. Trentesaux, and V. Prabhu. 2015. "Sustainability in Manufacturing Operations Scheduling: A State of the Art Review." *Journal of Manufacturing Systems* 37 (1): 126–140.

Glover, F. 1986. "Future Paths for Integer Programming and Links to Artificial Intelligence." *Computers & Operations Research* 13 (5): 533–549.

Gong, X., T. De Pessemier, L. Martens, and W. Joseph. 2019. "Energy- and Labor-aware Flexible Job Shop Scheduling Under Dynamic Electricity Pricing: A Many-objective Optimization Investigation." *Journal of Cleaner Production* 209: 1078–1094.

Gupta, J. N. D. 1988. "Two-stage, Hybrid Flowshop Scheduling Problem." *Journal of the Operational Research Society* 39 (4): 359–364.

He, Y., Y. F. Li, T. Wu, and J. W. Sutherland. 2015. "An Energy-responsive Optimization Method for Machine Tool Selection and Operation Sequence in Flexible Machining Job Shops." *Journal of Cleaner Production* 87: 245–254.

Ishibuchi, H., T. Yoshida, and T. Murata. 2003. "Balance Between Genetic Search and Local Search in Memetic Algorithms for Multiobjective Permutation Flowshop Scheduling." *IEEE Transactions on Evolutionary Computation* 7 (2): 204–223.

Ji, M., J. Y. Wang, and W. C. Lee. 2013. "Minimizing Resource Consumption on Uniform Parallel Machines with a Bound on Makespan." *Computers & Operations Research* 40: 2970–2974.

Jia, Z. H., Y. L. Zhang, J. Y. T. Leung, and K. Li. 2017. "Bi-criteria Ant Colony Optimization Algorithm for Minimizing Makespan and Energy Consumption on Parallel Batch Machines." *Applied Soft Computing* 55: 226–237.

Jiang, E. D., and L. Wang. 2019. "An Improved Multi-objective Evolutionary Algorithm Based on Decomposition for Energy-efficient Permutation Flow Shop Scheduling Problem with Sequence-dependent Setup Time." *International Journal of Production Research* 57 (6): 1756–1771.

Karhi, S., and D. Shabtay. 2017. "Single Machine Scheduling to Minimise Resource Consumption Cost with a Bound on Scheduling Plus Due Date Assignment Penalties." *International Journal of Production Research* 56 (9): 3080–3096.

Lee, S., B. D. Chung, H. W. Jeon, and J. Chang. 2017. "A Dynamic Control Approach for Energy-efficient Production Scheduling on a Single Machine Under Time-varying Electricity Pricing." *Journal of Cleaner Production* 165: 552–563.

Lee, C. Y., and G. L. Vairaktarakis. 1994. "Minimizing Makespan in Hybrid Flowshops." *Operations Research Letters* 16 (3): 149–158.

Lei, D. M., Y. L. Zheng, and X. P. Guo. 2017. "A Shuffled Frog-leaping Algorithm for Flexible Job Shop Scheduling with the Consideration of Energy Consumption." *International Journal of Production Research* 55 (11): 3126–3140.

Li, J. Q., H. Y. Sang, Y. Y. Han, C. G. Wang, and K. Z. Gao. 2018. "Efficient Multi-objective Optimization Algorithm for Hybrid Flow Shop Scheduling Problems with Setup Energy Consumptions." *Journal of Cleaner Production* 181: 584–598.

Li, K., X. Zhang, J. Y. T. Leung, and S. L. Yang. 2016. "Parallel Machine Scheduling Problems in Green Manufacturing Industry." *Journal of Manufacturing Systems* 38: 98–106.

Liu, C. H. 2014. "Approximate Trade-off Between Minimisation of Total Weighted Tardiness and Minimisation of Carbon Dioxide (CO_2) Emissions in Bi-criteria Batch Scheduling Problem." *International Journal of Computer Integrated Manufacturing* 27: 759–771.

Liu, Y., H. B. Dong, N. Lohse, S. Petrovic, and N. Gindy. 2014. "An Investigation Into Minimising Total Energy Consumption and Total Weighted Tardiness in Job Shops." *Journal of Cleaner Production* 65: 87–96.

Liu, Z. C., S. S. Guo, and L. Wang. 2019. “Integrated Green Scheduling Optimization of Flexible Job Shop and Crane Transportation Considering Comprehensive Energy Consumption.” *Journal of Cleaner Production* 211: 765–786.

Liu, M., X. N. Yang, F. Chu, J. T. Zhang, and C. B. Chu. 2018. “Energy-oriented Bi-objective Optimization for the Tempered Glass Scheduling.” *Omega*. doi:10.1016/j.omega.2018.11.004.

Liu, C. G., J. Yang, J. Lian, W. J. Li, S. Evans, and Y. Yin. 2014. “Sustainable Performance Oriented Operational Decision-making of Single Machine Systems with Deterministic Product Arrival Time.” *Journal of Cleaner Production* 85: 318–330.

Lu, C., L. Gao, X. Y. Li, J. Zheng, and W. Y. Gong. 2018. “A Multi-objective Approach to Welding Shop Scheduling for Makespan, Noise Pollution and Energy Consumption.” *Journal of Cleaner Production*196: 773–787.

Luo, H., B. Du, G. Q. Huang, H. P. Chen, and X. L. Li. 2013. “Hybrid Flow Shop Scheduling Considering Machine Electricity Consumption Cost.” *International Journal of Production Economics* 146: 423–439.

Mansouri, S. A., E. Aktas, and U. Besikci. 2016. “Green Scheduling of a Two-machine Flowshop: Trade-off Between Makespan and Energy Consumption.” *European Journal of Operational Research* 248 (3): 772–788.

May, G., B. Stahl, M. Taisch, and V. Prabhu. 2015. “Multi-objective Genetic Algorithm for Energy-efficient Job Shop Scheduling.” *International Journal of Production Research* 53 (23): 7071–7089.

Meng, L. L., C. Y. Zhang, X. Y. Shao, and Y. P. Ren. 2019. “MILP Models for Energy-aware Flexible Job Shop Scheduling Problem.” *Journal of Cleaner Production* 210: 710–723.

Meng, L. L., C. Y. Zhang, X. Y. Shao, Y. P. Ren, and C. Ren. 2019. “Mathematical Modelling and Optimisation of Energy-conscious Hybrid Flow Shop Scheduling Problem with Unrelated Parallel Machines.” *International Journal of Production Research* 57 (4): 1119–1145.

Merkert, L., I. Harjunkoski, A. Isaksson, S. Säynevirta, A. Saarela, and G. Sand. 2015. “Scheduling and Energy-industrial Challenges and Opportunities.” *Computers & Chemical Engineering* 72: 183–198.

Mokhtari, H., and A. Hasani. 2017. “An Energy-efficient Multi-objective Optimization for Flexible Job-shop Scheduling Problem.” *Computers & Chemical Engineering* 104: 339–352.

Moon, J. Y., and J. Park. 2014. “Smart Production Scheduling with Time-dependent and Machine-dependent Electricity Cost by Considering Distributed Energy Resources and Energy Storage.” *International Journal of Production Research* 52 (13): 3922–3939.

Mouzon, G., and M. B. Yildirim. 2008. “A Framework to Minimise Total Energy Consumption and Total Tardiness on a Single Machine.” *International Journal of Sustainable Engineering* 1 (2): 105–116.

Nasiri, M. M., M. Abdollahi, A. Rahbari, N. Salmanzadeh, and S. Salesi. 2018. “Minimizing the Energy Consumption and the Total Weighted Tardiness for the Flexible Flowshop Using NSGA-II and NRGA.” *Journal of Industrial and Systems Engineering* 11: 150–162.

Nouiri, M., A. Bekrar, and D. Trentesaux. 2018. “Towards Energy Efficient Scheduling and Rescheduling for Dynamic Flexible Job Shop Problem.” *IFAC-PapersOnLine* 51 (11): 1275–1280.

Rager, M., C. Gahm, and F. Denz. 2015. “Energy-oriented Scheduling Based on Evolutionary Algorithms.” *Computers & Operations Research* 54: 218–231.

Ribas, I., R. Leisten, and J. M. Framiñan. 2010. “Review and Classification of Hybrid Flow Shop Scheduling Problems From a Production System and a Solutions Procedure Perspective.” *Computers & Operations Research* 37 (8): 1439–1454.

Rubaiee, S., and M. B. Yildirim. 2019. “An Energy-aware Multiobjective Ant Colony Algorithm to Minimize Total Completion Time and Energy Cost on a Single-machine Preemptive Scheduling.” *Computers & Industrial Engineering* 127: 240–252.

Ruiz, R., and J. A. Vázquez-Rodriguez. 2010. “The Hybrid Flow Shop Scheduling Problem.” *European Journal of Operational Research* 205 (1): 1–18.

Safarzadeh, H., and S. T. A. Niaki. 2019. “Bi-objective Green Scheduling in Uniform Parallel Machine Environments.” *Journal of Cleaner Production* 217: 559–572.

Shrouf, F., J. Ordieres-Meré, A. García-Sánchez, and M. Ortega-Mier. 2014. “Optimizing the Production Scheduling of a Single Machine to Minimize Total Energy Consumption Costs.” *Journal of Cleaner Production* 67: 197–207.

Tacconi, L. 2000. “Biodiversity and Ecological Economic: Participation, Values and Resource Management.” *Biological Conservation* 107: 259–260.

Tan, M., B. Duan, and Y. X. Su. 2018. “Economic Batch Sizing and Scheduling on Parallel Machines Under Time-of-use Electricity Pricing.” *Operational Research* 18 (1): 105–122.

Tan, K. C., C. K. Goh, Y. J. Yang, and T. H. Lee. 2006. “Evolving Better Population Distribution and Exploration in Evolutionary Multi-objective Optimization.” *European Journal of Operational Research*171 (2): 463–495.

Tang, D. B., M. Dai, M. A. Salido, and A. Giret. 2016. “Energy-efficient Dynamic Scheduling for a Flexible Flow Shop Using An Improved Particle Swarm Optimization.” *Computers in Industry* 81: 82–95.

Veček, N., M. Mernik, B. Filipič, and M. vCrepinšek. 2016. “Parameter Tuning with Chess Rating System (CRS-Tuning) for Meta-heuristic Algorithms.” *Information Sciences* 372: 446–469.

Wang, H., Z. G. Jiang, Y. Wang, H. Zhang, and Y. H. Wang. 2018. “A Two-stage Optimization Method for Energy-saving Flexible Job-shop Scheduling Based on Energy Dynamic Characterization.” *Journal of Cleaner Production* 188: 575–588.

Wang, S. J., M. Liu, F. Chu, and C. B. Chu. 2016. “Bi-objective Optimization of a Single Machine Batch Scheduling Problem with Energy Cost Consideration.” *Journal of Cleaner Production* 137 (20): 1205–1215.

Wang, J. K., F. Qiao, F. Zhao, and J. W. Sutherland. 2016. "Batch Scheduling for Minimal Energy Consumption and Tardiness Under Uncertainties: A Heat Treatment Application." *CIRP Annals* 65 (1): 17–20.

Wang, Y. C., M. J. Wang, and S. C. Lin. 2017. "Selection of Cutting Conditions for Power Constrained Parallel Machine Scheduling." *Robotics and Computer-Integrated Manufacturing* 43: 105–110.

Wang, S. J., X. D. Wang, J. B. Yu, S. Ma, and M. Liu. 2018. "Bi-objective Identical Parallel Machine Scheduling to Minimize Total Energy Consumption and Makespan." *Journal of Cleaner Production* 193: 424–440.

Wu, X. Q., and A. D. Che. 2019. "A Memetic Differential Evolution Algorithm for Energy-efficient Parallel Machine Scheduling." *Omega* 82: 155–165.

Wu, X. L., and Y. J. Sun. 2018. "A Green Scheduling Algorithm for Flexible Job Shop with Energy-saving Measures." *Journal of Cleaner Production* 172: 3249–3264.

Xu, W. J., L. X. Tang, and E. N. Pistikopoulos. 2018. "Modeling and Solution for Steelmaking Scheduling with Batching Decisions and Energy Constraints." *Computers & Chemical Engineering* 116: 368–384.

Yan, J. H., L. Li, F. Zhao, F. Y. Zhang, and Q. L. Zhao. 2016. "A Multi-level Optimization Approach for Energy-efficient Flexible Flow Shop Scheduling." *Journal of Cleaner Production* 137: 1543–1552.

Yildirim, M. B., and G. Mouzon. 2012. "Single-Machine Sustainable Production Planning to Minimize Total Energy Consumption and Total Completion Time Using a Multiple Objective Genetic Algorithm." *IEEE Transactions on Engineering Management* 59 (4): 585–597.

Zeng, Y. Z., A. D. Che, and X. Q. Wu. 2018. "Bi-objective Scheduling on Uniform Parallel Machines Considering Electricity Cost." *Engineering Optimization* 50 (1): 19–36.

Zeng, Z. Q., M. N. Hong, Y. Man, J. G. Li, Y. Z. Zhang, and H. B. Liu. 2018. "Multi-object Optimization of Flexible Flow Shop Scheduling with Batch Process–consideration Total Electricity Consumption and Material Wastage." *Journal of Cleaner Production* 183: 925–939.

Zhang, S. B. H., A. D. Che, X. Q. Wu, and C. B. Chu. 2018. "Improved Mixed-integer Linear Programming Model and Heuristics for Bi-objective Single-machine Batch Scheduling with Energy Cost Consideration." *Engineering Optimization* 50 (8): 1380–1394.

Zhang, R., and R. Chiong. 2016. "Solving the Energy-efficient Job Shop Scheduling Problem: A Multi-objective Genetic Algorithm with Enhanced Local Search for Minimizing the Total Weighted Tardiness and Total Energy Consumption." *Journal of Cleaner Production* 112: 3361–3375.

Zhang, H., F. Zhao, K. Fang, and J. W. Sutherland. 2014. "Energy-conscious Flow Shop Scheduling Under Time-of-use Electricity Tariffs." *CIRP Annals* 63 (1): 37–40.

Zhao, S. N., I. E. Grossmann, and L. X. Tang. 2018. "Integrated Scheduling of Rolling Sector in Steel Production with Consideration of Energy Consumption Under Time-of-use Electricity Prices." *Computers & Chemical Engineering* 111: 55–65.

Zhou, S. C., X. L. Li, N. Du, Y. Pang, and H. P. Chen. 2018. "A Multi-objective Differential Evolution Algorithm for Parallel Batch Processing Machine Scheduling Considering Electricity Consumption Cost." *Computers & Operations Research* 96: 55–68.

Appendix

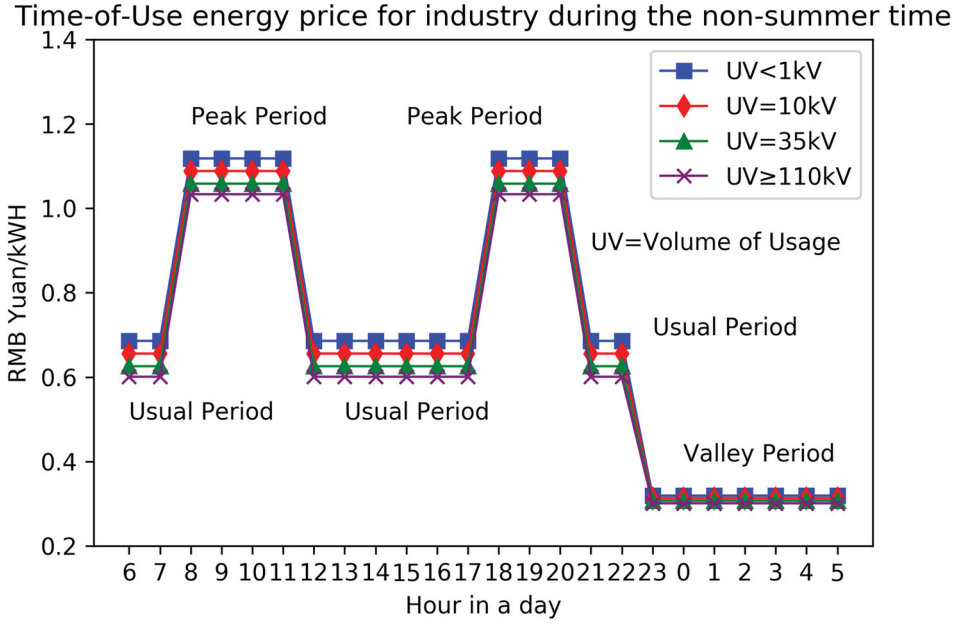

Figure A.1. Electricity price for industry during the non-summer time in Shanghai, China (Source: Shanghai Electricity Price, 2018, www.sh.sgcc.com.cn. Accessed on 19 February, 2019).

Table A.1. The detailed machine-related information of the case (unit: kW).

Stage	*Machines*	$ECR_{working}$	ECR_{idle}	$ECR_{turnoff}$
1	M_1	40	2	0
	M_2	58	3	0
	M_3	58	3	0
	M_4	50	2.5	0
	M_5	40	2	0
2	*Furnace*	110	20	0

Table A.2. The detailed processing-related information of the case (processing time unit: 10 min).

	*Stage*1					*Stage*2			*Stage*1					*Stage*2	
Jobs	PT_{M1}	PT_{M2}	PT_{M3}	PT_{M4}	PT_{M5}	$PT_{Furnace}$	*Size*	*Jobs*	PT_{M1}	PT_{M2}	PT_{M3}	PT_{M4}	PT_{M5}	$PT_{Furnace}$	*Size*
1	2	2	2	2	\	8	6	26	2	2	2	2	\	8	6
2	8	\	8	\	8	4	8	27	8	\	8	\	8	4	8
3	\	4	\	4	4	10	2	28	3	3	3	\	3	2	5
4	\	4	\	4	4	10	2	29	2	2	2	2	\	8	6
5	10	\	10	10	10	8	5	30	10	\	10	10	10	8	5
6	10	\	10	10	10	8	5	31	8	\	8	\	8	4	8
7	3	3	3	\	3	2	5	32	3	3	3	\	3	2	5
8	2	2	2	2	\	8	6	33	3	3	3	\	3	2	5
9	8	\	8	\	8	4	8	34	3	3	3	\	3	2	5
10	3	3	3	\	3	2	5	35	8	\	8	\	8	4	8
11	3	3	3	\	3	2	5	36	3	3	3	\	3	2	5
12	2	2	2	2	\	8	6	37	3	3	3	\	3	2	5
13	2	2	2	2	\	8	6	38	2	2	2	2	\	8	6
14	2	2	2	2	\	8	6	39	2	2	2	2	\	8	6
15	\	4	\	4	4	10	2	40	8	\	8	\	8	4	8
16	3	3	3	\	3	2	5	41	8	\	8	\	8	4	8
17	3	3	3	\	3	2	5	42	8	\	8	\	8	4	8
18	3	3	3	\	3	2	5	43	3	3	3	\	3	2	5
19	8	\	8	\	8	4	8	44	\	4	\	4	4	10	2
20	2	2	2	2	\	8	6	45	3	3	3	\	3	2	5
21	3	3	3	\	3	2	5	46	2	2	2	2	\	8	6
22	10	\	10	10	10	8	5	47	8	\	8	\	8	4	8
23	3	3	3	\	3	2	5	48	8	\	8	\	8	4	8
24	8	\	8	\	8	4	8	49	3	3	3	\	3	2	5
25	\	4	\	4	4	10	2	50	10	\	10	10	10	8	5